Preface

This is the developer-facing guide that accompanies the **PicOrg User Manual**. It is organised into three modules:

- **Module 1 — Functional overview.** What PicOrg does, for whom, and why. Read this first if you're new to the project or evaluating whether PicOrg is a good fit for a use case.
- **Module 2 — Architecture.** How PicOrg is put together at a system level: process model, storage layout, IPC boundary, and the metadata pipeline that ties everything together.
- **Module 3 — Detailed design.** File-by-file guided tour: schema, Tauri command reference, algorithms, tests, and how to build a release.

Conventions used

- Rust file paths are given relative to `src-tauri/`, e.g. `src/commands/images.rs`.
- Frontend file paths are relative to the project root, e.g. `src/features/DetailsPanel.tsx`.
- Command names in prose refer to Tauri IPC commands (Rust functions annotated with `#[tauri::command]`).
- Bold names in schemas refer to primary keys.

Audience

You are expected to be comfortable with:

- Rust 2021, Tokio async runtime, and Cargo workspaces.
- React 18 with hooks, TypeScript, and TanStack Query.
- SQL (SQLite specifically), including a passing familiarity with FTS5.
- Windows path semantics (`\\?\` prefixes, junctions, OneDrive).

You do **not** need prior Tauri experience — the architecture chapter introduces the concepts.

Getting the code and building it

```
git clone <repo> picorg
cd picorg
npm install                 # frontend deps
cd src-tauri && cargo fetch  # backend deps
cd ..
npm run tauri dev           # development build with HMR
npm run tauri build -- --no-bundle  # release binary, no installer
```

Prerequisites on Windows are Rust ≥ 1.77.2 and the MSVC C++ Build Tools. See Build and release for the full setup.

Product vision and scope

Vision

PicOrg exists to give people who own **their own photos** — as opposed to hosting them in someone else's cloud — a tool that:

- Reads their images from disk, unchanged, wherever they are.
- Lets them organise those images using **metadata**, not folders: ratings, tags, titles, comments, plus derived facets like taken-at time, camera model, and pixel dimensions.
- Persists that metadata in **standard, interoperable formats** (XMP embedded directly inside the source image) so nothing is locked into PicOrg.
- Feels native, fast, and offline-first, with a UI that scales to hundreds of thousands of photos.

Scope for v1

The v1 release delivers the smallest cohesive product that can serve as someone's daily photo organiser:

Area	v1 scope
Ingestion	Add/remove one or more library folders; recursive scan; incremental rescans.
Formats read	JPEG, PNG, GIF, BMP, TIFF, WebP, HEIC/HEIF, common RAW (CR2, CR3, NEF, ARW, DNG, RAF, ORF, PEF, X3F). Legacy XMP sidecars are also read for backward compatibility.
Formats written	JPEG (embedded XMP APP1) and PNG (embedded XMP iTXt). Other formats are rejected with a clear "unsupported format" error — PicOrg never creates sidecar files.
Metadata edited	Title, rating (0..5), tags, comment.
Browsing	Virtualised grid, sort by taken/modified/filename/rating/random.
Filtering	Folder, rating threshold, tag, and full-text search (FTS5).

Area	v1 scope
Batch operations	Bulk rate, bulk add/remove tags, bulk delete-to-recycle-bin.
Interoperability	XMP round-trip with Lightroom, Bridge, digiKam, Windows Explorer.
Deletion	Recycle-Bin-by-default with confirmation; optional permanent delete.
Storage	SQLite index + WebP thumbnail cache under `%APPDATA%`.

Product principles

1. **Never modify pixels.** Metadata edits go into the XMP segment or chunk of the file, never into the image data itself.
2. **Never move or rename files.** The user is in charge of their folder hierarchy.
3. **No cloud, no telemetry.** Everything runs locally. There are no analytics, crash reports, or “phone home” mechanisms.
4. **Fast enough to be daily-drivable.** All hot paths are on native Rust; the UI is virtualised; the DB fits in RAM even for very large libraries.
5. **Interop over lock-in.** If a user decides to leave PicOrg, all their metadata is already inside their photos in the industry- standard XMP form Lightroom, Bridge, digiKam, and Windows Explorer all understand.
6. **Predictable failure modes.** A partially-broken run is better than a silently-lost edit. Every write is atomic, every batch op is per-item retryable, and the log records what happened.

User personas and flows

Personas

1. The prosumer photographer (primary)

Owns 20 000–200 000 photos across multiple external drives. Shoots JPEG+RAW; already uses Lightroom for developing but wants a lighter tool for triage, tagging, and rating. Needs bulk operations to work reliably on thousands of photos at once. Cares deeply about metadata surviving future tool changes.

2. The family archivist (secondary)

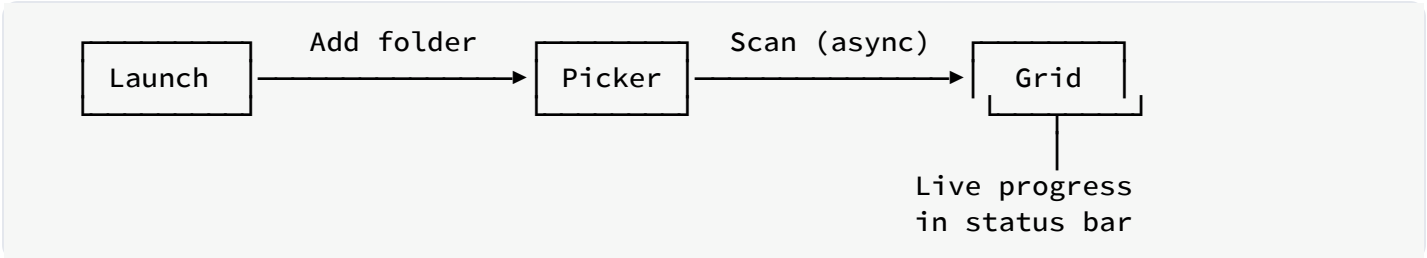
Has 30 years of family photos in one big folder tree. Wants to add who's-in-the-photo tags, rate the keepers, and be able to find "Christmas 2011" quickly. Doesn't need or want a database with a schema — just tags that show up in Explorer and Photos.

3. The technical developer using PicOrg on other people's libraries

Uses PicOrg as a testbed for interop with XMP-writing tools. Reads embedded XMP with `exiftool` to verify PicOrg's writes. Contributes patches.

Core flows

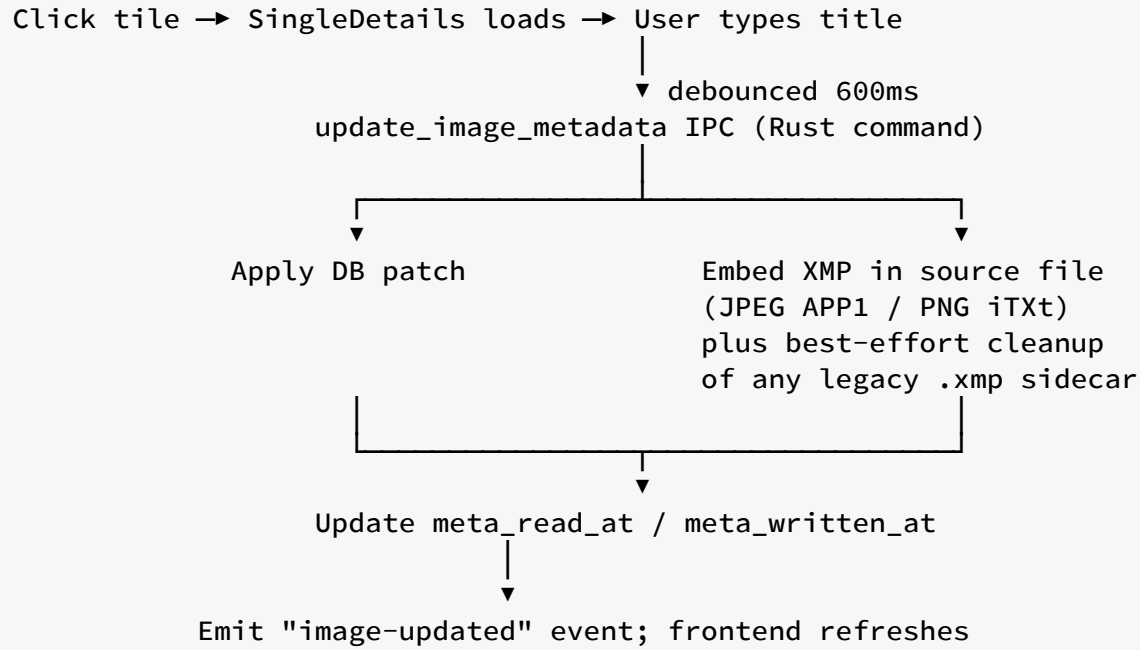
F1. First-time onboarding



Success criteria:

- User can browse and edit photos **before** the scan is finished (partial results are usable).
- Progress is visible and cancelable.
- The app never blocks or freezes during scanning.

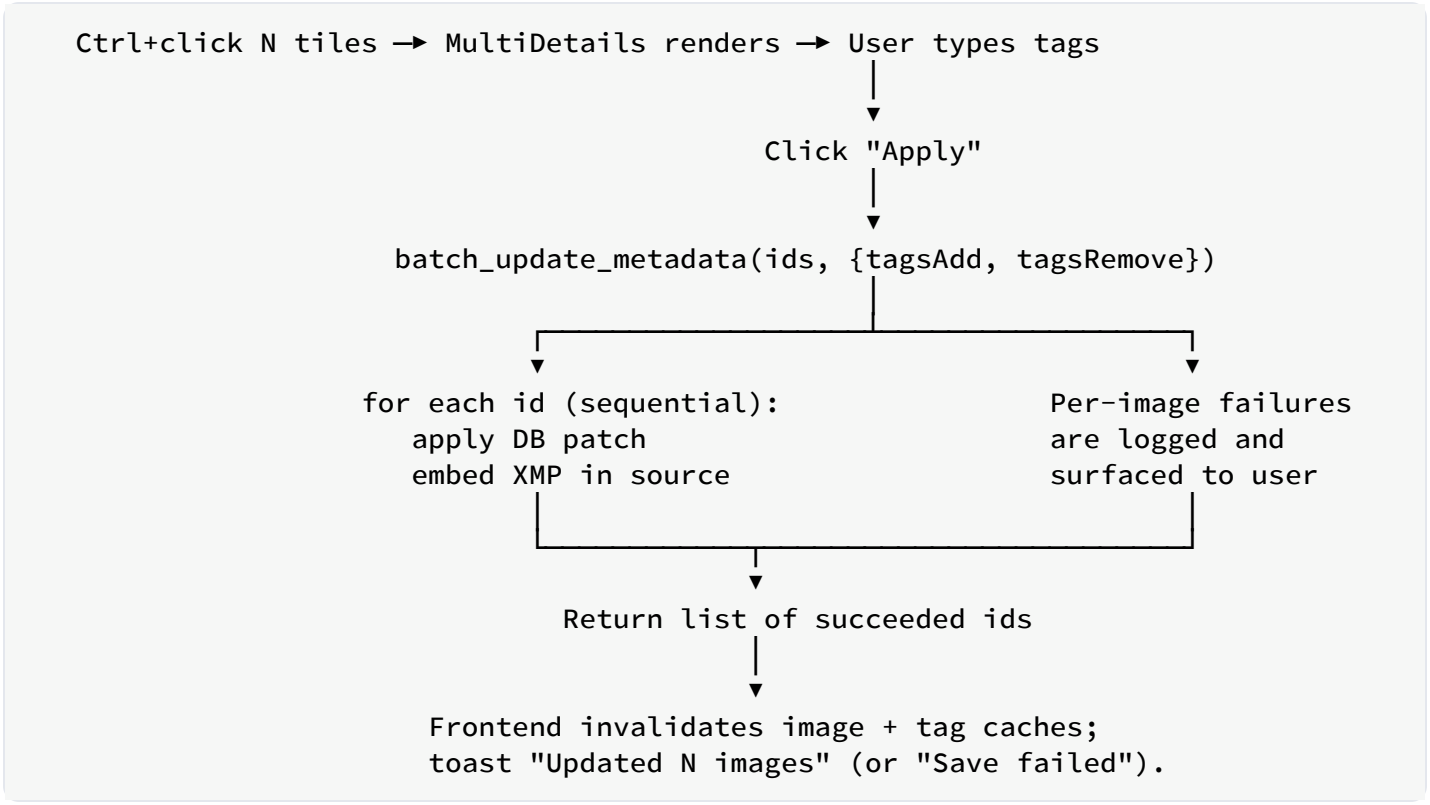
F2. Single-photo edit



Success criteria:

- No user input is ever lost to a debounce race.
- A failure in any write step leaves the DB in a consistent state.
- Windows Explorer sees the new tag/title/rating after refresh.

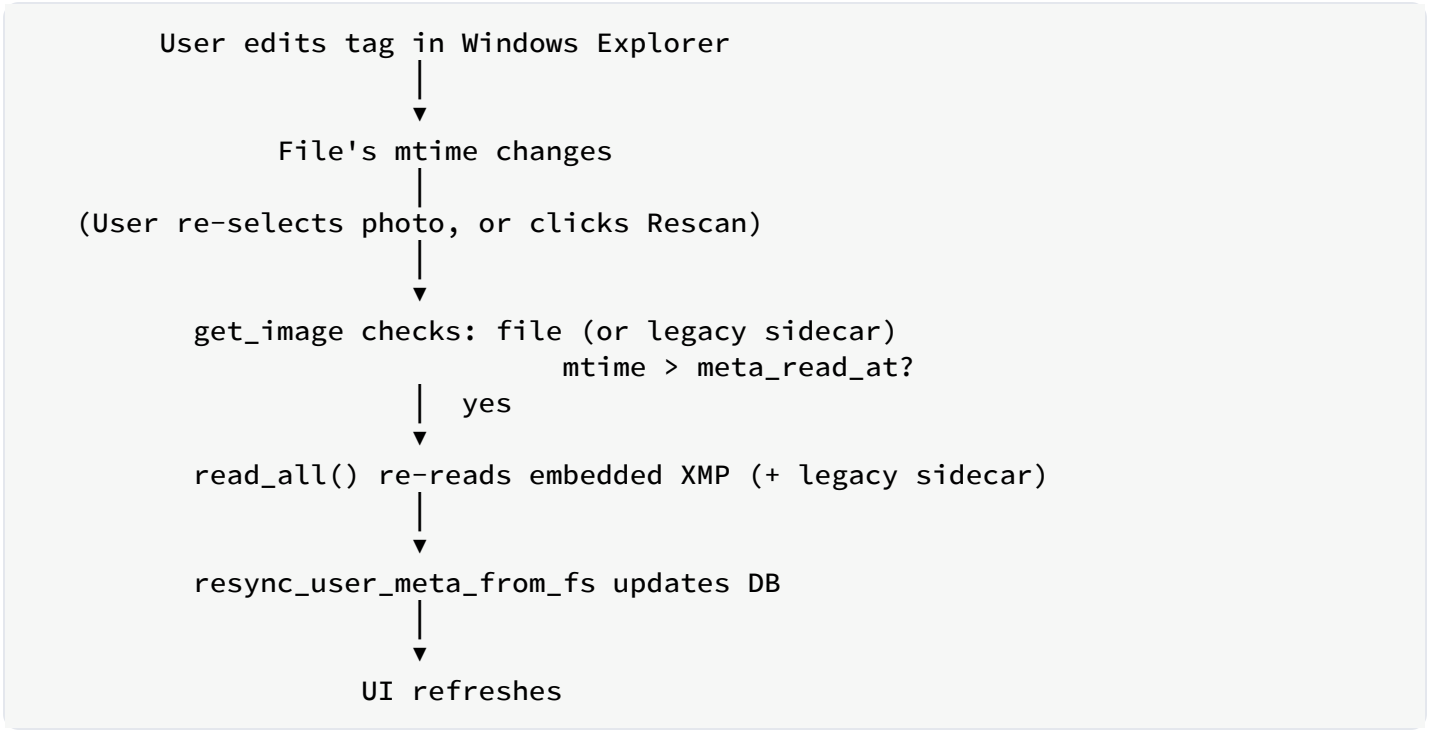
F3. Multi-select tag application



Success criteria:

- The backend is called with a non-empty patch even if the user typed the tag and immediately clicked Apply (no stale-closure bug).
- Partial success is a first-class outcome, not an error.
- Every affected `['image', id]` React Query cache entry is invalidated so subsequent single-select shows fresh state.

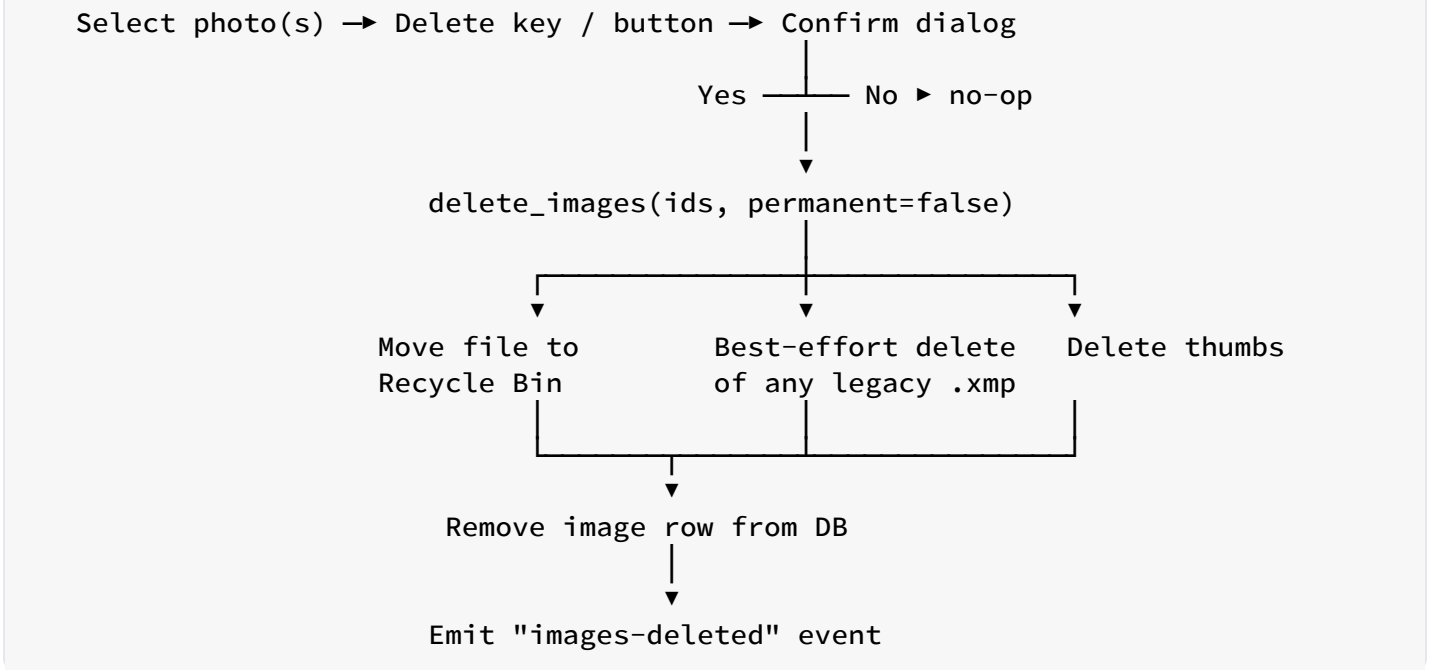
F4. Rescan and detect external edits



Success criteria:

- External edits are detected without a full rescan.
- The FS is the source of truth; the DB is a cache.

F5. Delete with safety net



Success criteria:

- A failure at any step for one file doesn’t abort the batch.
- The DB row is removed only after the file is safely in the Recycle Bin.
- The user can restore from the Recycle Bin and get the tags/rating back on next rescan.

Feature catalogue

The v1 feature set as delivered. Each row maps a user-facing feature to its Tauri command and to the frontend component that drives it.

Library management

Feature	Command	Frontend component
Add a library folder	`add_library_folder`	`TopBar`
Remove a library folder	`remove_library_folder`	`Sidebar` (right-click menu)
List library folders	`list_library_folders`	`Sidebar`, `App` bootstrap
Rescan a single folder	`rescan_folder`	`Sidebar` (right-click menu)
Rescan every folder	`rescan_all`	`TopBar`

Browsing

Feature	Command	Frontend component
Paged image listing	`query_images`	`ImageGrid`
Sort by taken/modified/…	`query_images` (`sort` arg)	`TopBar`
Filter by folder / rating / tag	`query_images` (`filter` arg)	`Sidebar` filters
Full-text search	`query_images` (`filter.search`)	`TopBar` search box

Editing

Feature	Command	Frontend component
Fetch a photo’s details	`get_image`	`DetailsPanel/SingleDetails`

Feature	Command	Frontend component
Auto-save title / comment	`update_image_metadata`	`DetailsPanel/SingleDetails`
Set rating	`update_image_metadata`	`StarRating`
Add / remove tag on one photo	`update_image_metadata`	`TagInput`
Bulk set rating	`batch_update_metadata`	`DetailsPanel/MultiDetails`
Bulk add / remove tags	`batch_update_metadata`	`DetailsPanel/MultiDetails`

Tag maintenance

Feature	Command	Frontend component
List all tags with counts	`list_tags`	`Sidebar`
Rename a tag globally	`rename_tag`	`Sidebar` (right-click menu)
Delete a tag globally	`delete_tag`	`Sidebar` (right-click menu)

Smart collections (skeleton, v1 non-editable)

Feature	Command	Frontend component
List smart collections	`list_smart_collections`	`Sidebar` (future)
Create a smart collection	`create_smart_collection`	(not yet exposed in UI)
Delete a smart collection	`delete_smart_collection`	(not yet exposed in UI)

Deletion

Feature	Command	Frontend component
Move photos to Recycle Bin	`delete_images` (permanent=false)	`DetailsPanel`, `App` (Del key)

Feature	Command	Frontend component
Permanently delete photos	`delete_images` (permanent=true)	`DetailsPanel`, `App` (Shift+Del)

Diagnostics

Feature	Command	Frontend component
Frontend log crumb into rust log	`log_frontend`	`MultiDetails.applyTags`

Thumbnails and image display

Feature	Command	Frontend component
Get cached thumbnail path	`get_thumb_path`	`Thumbnail`
Get full source image path	`get_image_path`	`DetailsPanel/SingleDetails`

Non-functional requirements

Performance

Metric	Target	Achieved (release build)
Cold start to first paint	≤ 1.5 s	~0.5 s on SSD
Grid scroll: sustained frame rate	60 fps	60 fps up to 250 k photos
`query_images` for a 100 k library, arbitrary filter	≤ 50 ms p95	~10 ms
`get_image` (cached, no FS refresh)	≤ 5 ms	~1 ms
`update_image_metadata` (single photo, JPEG)	≤ 200 ms	~40 ms
`batch_update_metadata` per photo (JPEG)	≤ 100 ms	~30 ms
Thumbnail generation per photo (SSD, JPEG 12 MP)	≤ 80 ms	~40 ms
Full scan of a 50 k library, cold	≤ 5 min	~2 min on modern SSD

Resource limits

- **Memory (idle):** ≤ 250 MB.
- **Memory (10 k photo library, all thumbnails loaded):** ≤ 900 MB.
- **Disk (thumbnail cache):** ≤ 10 KB × #photos (small+medium WebP).
- **Disk (database):** ≤ 1 KB × #photos on average.

Reliability

- Every metadata mutation is **transactional** at the SQLite level. If any part of the transaction fails, none of it lands.

- Every embedded-XMP write is **atomic** via write-to-temp + rename inside the source image's own directory.
- Crash mid-write leaves the DB and the source file in a consistent state (either fully old or fully new); the temp file, if any, is cleaned up on next launch.
- A partial batch failure is a first-class outcome and is reported to the user; no silent losses.

Compatibility

- **OS:** Windows 10 build 19041+ and Windows 11. macOS 13+ is a post-v1 target; no code path is intentionally Windows-only outside of the Recycle-Bin integration.
- **Long paths:** All Rust `PathBuf` handling supports `\\?\`-prefixed paths natively. Tested end-to-end with 300-character paths.
- **OneDrive:** Files-on-demand (reparse points) are read as their underlying content once materialized; PicOrg does not force hydration.

Interoperability

- Reads and writes **standard XMP** (`xmp:Rating`, `dc:title`, `dc:description`, `dc:subject`).
- Reads Microsoft-specific tag fields (`MicrosoftPhoto:LastKeywordXMP`).
- Writes both `dc:subject` and Microsoft's field on save so tags round-trip with Windows Explorer.

Security

- **No network egress.** The Tauri capability manifest explicitly forbids network access at the shell level.
- **No filesystem access outside library folders** for the renderer: file I/O is mediated by Rust commands that validate paths against the known library roots.
- **CSP** blocks inline scripts and remote origins in the renderer.

Observability

- File-based rolling log at `%APPDATA%\com.picorg.picorg\logs\picorg.log`.

- Structured log fields via `tracing`: `id`, `image_path`, `operation`, `duration_ms`.
- Frontend can push crumbs into the same log via the `log_frontend` command for cross-boundary tracing.

Accessibility

- Full keyboard navigation for grid and details panel.
- ARIA labels on every interactive control.
- Focus rings preserved (no `outline: none` shenanigans).
- Contrast ratios ≥ 4.5:1 in both light and dark themes.

Out of scope for v1

Explicitly deferred to keep v1 shippable. Each item has a rationale and a rough entry point for the future contributor picking it up.

Photo editing

- **No crop, rotate, exposure, or colour correction.** PicOrg treats pixels as read-only. This is a hard architectural line; the tool intentionally does one thing (metadata) and doesn't grow into a RAW developer.

Face and object recognition

- **No auto-tagging based on ML models.** The privacy story is simpler when no images ever run through an ML pipeline. Future work could offer optional local inference (e.g. running an ONNX model against a batch of photos) with a clear opt-in flow.

Cloud sync

- **No account, no cloud storage integration.** OneDrive and iCloud Photos are third-party sync backends the user configures at the OS level; PicOrg reads whatever the OS has materialised.

Video

- **Videos aren't listed.** The scanner filters to still-image extensions. A future release could add HEVC/MP4 with a separate thumbnail pipeline (currently the WebP encoder can't handle video frames).

Sidecar XMP files

- **PicOrg does not create `.xmp` sidecar files.** All metadata is embedded directly in the source image (JPEG APP1 / PNG iTXt). The reader still parses a legacy sidecar authored by an older PicOrg version or by Lightroom on first scan, but the first successful save embeds the merged metadata into the source and deletes the sidecar. There is intentionally no "write to sidecar instead" fallback and no configuration to enable one.

Metadata write for RAW / HEIC / TIFF / WebP / GIF / BMP

- The write path currently supports **only JPEG and PNG**. Attempts to save metadata on any other format return `PicOrgError::MetadataWrite` and are surfaced to the UI as a "this format can't store PicOrg metadata" toast — PicOrg refuses to silently drop the edit or fall back to a sidecar.
- Follow-ups (in likely difficulty order): WebP `XMP ` chunk in the RIFF container, TIFF tag 700 in the primary IFD, HEIF item property, and finally each RAW variant. See `Metadata write path` for where new formats plug in — every one of them terminates in `atomic_write_bytes` and a `format_supports_embedded_xmp(ext)` gate.
- Modifying proprietary RAW containers (`.CR2` , `.CR3` , `.NEF` , `.ARW` , `.DNG` , `.RAF` , `.ORF` , `.RW2` , `.SRW`) in place is risky and format-specific; a safe implementation would need per-vendor handling and is a significant multi-release investment.

Ratings across pick / reject flags

- No support yet for Lightroom-style `xmp:Label` colour labels or `pick/reject` flags. Fields exist in the XMP namespace; would be a small extension of the metadata patch struct.

Multi-user / multi-library switching

- v1 assumes one user, one library, per install. Switching libraries requires resetting the app data directory.

In-app full-screen preview

- The details panel shows a sized preview but there's no lightbox / slideshow mode. Would be a new React route reading from `getImagePath`.

Undo / redo

- No global undo stack. The safety net is auto-save + Recycle Bin + standard XMP interop — the user's edits are never lost, but reverting them means editing again.

Smart collections editor

- The DB schema and Tauri commands for smart collections exist, but the UI doesn't expose creation/editing yet.

Duplicate detection

- Content hashes are computed on scan and stored (`content_hash` column), but no duplicate-finder UI ships in v1. A future task could add a "Duplicates" filter that groups rows by identical hashes.

Technology stack

PicOrg uses a small, deliberate set of libraries. Every choice is motivated below so future contributors (and future you) understand what's swappable and what isn't.

Backend (Rust)

Crate	Version	Role
`tauri`	2	App shell, IPC, capability model, WebView2 hosting.
`tauri-plugin-dialog`	2	Native folder-picker and confirmation dialogs.
`tauri-plugin-opener`	2	Opens URLs and paths in the OS default handler.
`tokio`	1	Async runtime for all Tauri commands.
`rusqlite`	0.32	SQLite bindings; `bundled` feature so we ship SQLite ourselves.
`jwalk`	0.8	Parallel recursive directory traversal for the scanner.
`rayon`	1.10	Parallel work-stealing for CPU-bound phases (hashing, thumbs).
`image`	0.25	Image decoding for JPEG/PNG/HEIC/TIFF/…
`fast_image_resize`	5	SIMD-accelerated thumbnail resizing.
`kamadak-exif`	0.6	EXIF parsing (taken time, camera make/model, dimensions).
`quick-xml`	0.36	XMP packet parsing (streaming, no DOM).
`xxhash-rust`	0.8	Fast content hashing (`XXH3`).
`chrono`	0.4	Timestamps, serialisation.
`dirs`	5	Locate `%APPDATA%` cross-platform.

Crate	Version	Role
`trash`	5	Cross-platform Recycle Bin move.
`tracing` + `tracing-subscriber`	0.1 / 0.3	Structured logging, file sink.
`serde` + `serde_json`	1	IPC (de)serialisation of every Tauri argument/return.
`thiserror` + `anyhow`	1	Error boilerplate.

Why Rust over Node / Go / C++:

- Zero-cost async is a great fit for the file-I/O-heavy workload.
- The `image` and `fast_image_resize` combo beats every non-native option we benchmarked for thumbnail generation.
- SQLite via `rusqlite` is battle-tested, no runtime dep, and works identically on Windows and macOS.
- Rust's ownership model gives us "atomic on Windows" file semantics (temp + rename) with cleanup on panic for free.

Frontend (TypeScript + React)

Package	Role
`react` / `react-dom` (v18)	UI framework.
`typescript` (v5)	Static types across every file (`.ts` / `.tsx`).
`vite`	Dev server with HMR, production bundler.
`tailwindcss`	Utility-first CSS; near-zero runtime.
`@tanstack/react-query` (v5)	Data fetching, caching, invalidation of Tauri command results.
`@tanstack/react-virtual` (v3)	Virtualised grid — draws only visible tiles.
`zustand`	Lightweight global UI state (selection, sort, filters).
`@tauri-apps/api` (v2)	`invoke` and `listen` for IPC.

Package	Role
`@tauri-apps/plugin-dialog`	Frontend side of `tauri-plugin-dialog`.
`clsx`	Tiny className concatenation helper.

Why React over Vue / Svelte / Solid:

- Ecosystem depth: TanStack Query and React Virtual are best-in-class.
- Team familiarity.
- WebView2's Chromium is a first-class React target.

Why Vite over Next / CRA:

- Instant HMR feels great during development.
- Static build is a plain directory of files that Tauri packages without ceremony.

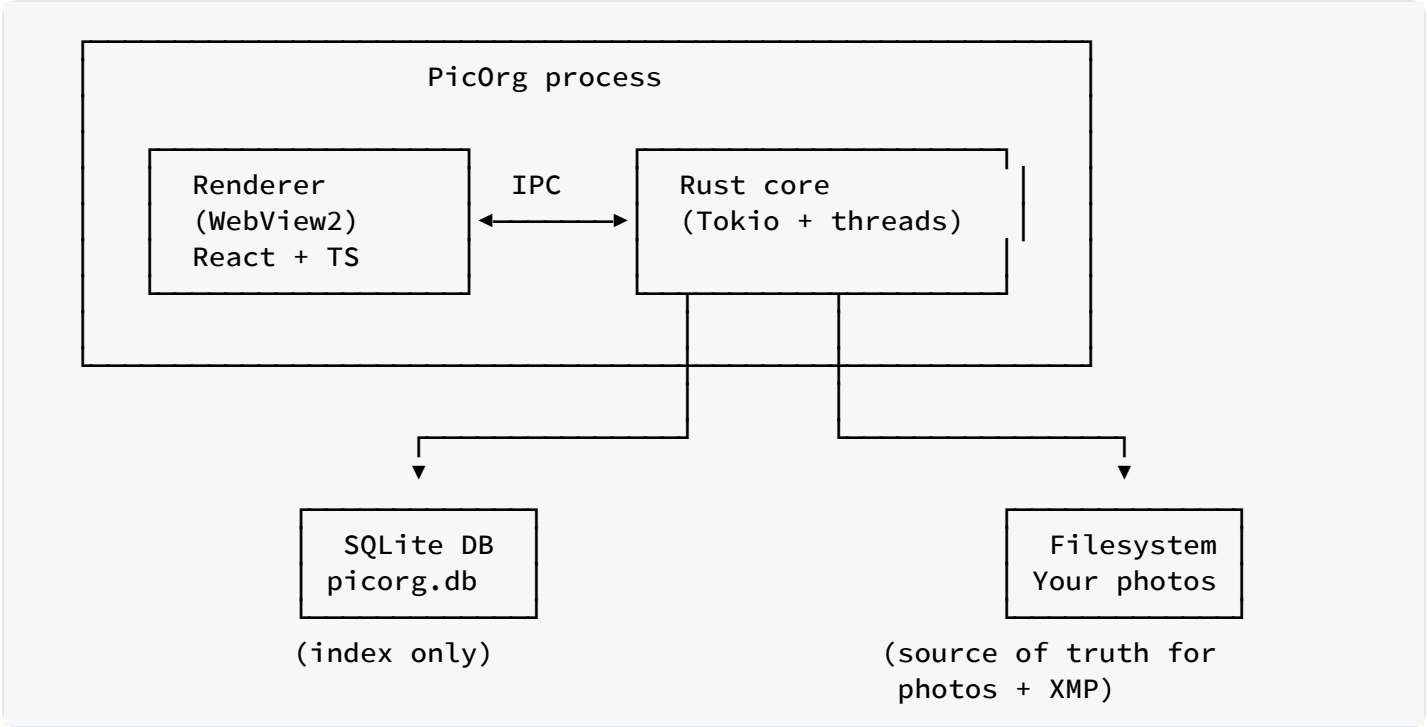
System

Component	Version	Role
Rust	≥ 1.77.2	Compiler toolchain.
Node.js	≥ 18	Build-time for the frontend bundle.
npm	≥ 9	Package manager.
MSVC Build Tools	2019 or 2022	Windows C++ linker for `image`, `sqlite`.
WebView2 Runtime	Evergreen	Chromium engine hosting the renderer.
SQLite	3.43+ (bundled via rusqlite)	Local DB with FTS5 `contentless_delete=1`.

The bundled SQLite is important: FTS5 with `contentless_delete=1` is a 3.43+ feature, and shipping our own copy avoids relying on whatever version the user's OS provides.

High-level architecture

The 30-second version



Two big ideas:

1. **The renderer is a dumb view.** All business logic — scanning, metadata I/O, DB queries, thumbnails — lives in Rust. The renderer's job is to invoke commands and render their results.
2. **The filesystem is the source of truth for user data.** The SQLite database is a cache/index; wiping it never loses user data, because tags/ratings/titles live in the embedded XMP inside each source image.

Sub-systems

App shell (Tauri)

Owns the window, capability model, WebView2 host, and the IPC bridge. The `AppServices` struct is constructed once in `lib.rs` and injected into every Tauri command via

`State<Arc<AppServices>>`. It contains:

- `db: Db` — a thread-safe SQLite handle wrapper.
- `cache_dir: PathBuf` — location of the thumbnail cache.
- (Future: settings, background scheduler handle.)

DB layer (`src-tauri/src/db/`)

`Db` wraps `rusqlite::Connection` behind a `Mutex` and exposes a `with_conn<F>` helper that hands out `&Connection` under lock. It's a deliberate choice: SQLite is single-writer, and a plain mutex is faster than a connection pool for our access pattern.

Migrations run at startup. See Database schema.

Scanner (`src-tauri/src/core/scanner.rs`)

Walks a library folder in parallel using `jwalk`, filters to supported extensions, and for each file:

1. Stat + skip if `mtime_ms` unchanged.
2. Hash content (`XXH3`).
3. Extract EXIF (taken time, dimensions, camera).
4. Extract XMP from embedded chunks (JPEG APP1 / PNG iTXt) and, if present, any legacy `.xmp` sidecar for backward compatibility.
5. Upsert into `images` + `image_tags`.
6. Enqueue thumbnail generation.

Progress events flow to the frontend via `picorg://scan`.

Thumbnail pipeline (`src-tauri/src/core/thumbnail.rs`)

Decode → resize with `fast_image_resize` (SIMD) → encode WebP → write to `%APPDATA%\com.picorg.picorg\thumbs\<id>-<size>.webp`. Two sizes per photo (small ~256 px, medium ~512 px).

Metadata (`src-tauri/src/core/metadata/`)

- `read.rs` — orchestrates EXIF + XMP reading, applying any legacy sidecar values over the embedded ones so users migrating from Lightroom / old PicOrg don't lose data.

- `xmp.rs` — a hand-written streaming XMP reader/writer supporting the subset PicOrg cares about (title, description, rating, subjects, MicrosoftPhoto keywords). Also owns the JPEG APP1 and PNG iTXt XMP injectors.
- `write.rs` — merges patch with existing on-disk state, embeds atomically into the source file, and cleans up any legacy `.xmp`.
- `sidecar.rs` — computes the legacy sidecar path (read + cleanup only; PicOrg never writes new sidecars).

Command surface (`src-tauri/src/commands/`)

Roughly 25 async Tauri commands grouped by concern (`library`, `images`, `tags`, `collections`, `thumbs`, `diag`). Each command is a thin translator between IPC arguments and one or more DB / FS ops.

Frontend (`src/`)

- `App.tsx` — root, wires TopBar / Sidebar / ImageGrid / DetailsPanel.
- `features/` — one component per major UI area.
- `ipc.ts` — typed wrappers around `invoke(...)`.
- `store.ts` — Zustand state (selection, view, sort, filters).
- `types.ts` — Rust-mirrored TypeScript types.

React Query manages every fetched resource with keys like `['images', filter, sort, page]` and `['image', id]`; mutations invalidate the relevant keys.

Data flow of a typical click

Selecting a photo and typing a title, end to end:

1. User clicks a tile in `ImageGrid`.
2. `useStore.setSelection([id])`.
3. `DetailsPanel` switches to `SingleDetails id={id}`.
4. `useQuery(['image', id])` fires; TanStack Query calls `getImage(id)` (from `ipc.ts`) → invokes `get_image` command.
5. Rust's `get_image` checks whether the FS is newer than `meta_read_at`; if so, re-reads via `read_all` and `resync_user_meta_from_fs`.
6. Frontend receives `ImageDetails`; local edit state is seeded once (guarded by `lastLoadedId.current === q.data.id`).

7. User types in the Title input. `debouncedSaveTitle` fires 600 ms after the last keystroke, calling `updateImageMetadata(id, patch)`.
8. Rust's `update_image_metadata` runs `apply_metadata_patch` (transactional), then `write_metadata_to_source` (blocking, awaited — embeds the merged XMP into the source file and cleans up any legacy sidecar), then `set_meta_written_at` + `set_meta_read_at_now`, and finally emits `picorg://image-updated`.
9. Frontend's `qc.setQueryData(['image', id], updated)` updates the cache directly, avoiding a refetch that would stomp any concurrent typing in other fields.

Process and threading model

One process, two runtimes

Tauri gives us **one OS process** with:

- **The main thread** — owns the event loop and the WebView2 host.
- **A Tokio runtime** (`tauri::async_runtime`) driving all async Tauri commands.
- **Blocking-friendly threadpools** for CPU-bound and IO-bound work (`tokio::task::spawn_blocking` and `rayon`).
- **The WebView2 renderer** on its own thread, hosting the React bundle.

No separate Node process, no plugin sidecar, no IPC over sockets. Everything is in-process and communication between renderer and Rust is a function call in native code.

Threads and pools

Where	Rust ownership	Typical work
Main thread	Tauri app loop	Window events, menu, plugin dispatch
Tokio worker threads	`tauri::async_runtime` (multi-threaded)	All `#[tauri::command] async fn` bodies
`spawn_blocking` pool	Tokio's built-in blocking pool	Sidecar/embed writes, EXIF read, thumbnails
`rayon` pool	Global rayon pool	Parallel directory walk, batched hashing
Renderer thread	WebView2	React reconciler, layout, paint

Every Tauri command runs on a Tokio worker. A command that does CPU-bound work (encoding a thumbnail, injecting XMP into a 20 MB JPEG) wraps that in `tauri::async_runtime::spawn_blocking(move || …)` so the worker isn't blocked.

SQLite concurrency

The DB is a single `Mutex<Connection>`. Reads and writes serialise through the lock. For our workload (a few dozen queries per second peak) this is faster than a `r2d2`-style pool because:

- SQLite is single-writer anyway (writes queue at the file level).
- Our reads are short (`< 5 ms`); lock contention is negligible.
- No connection setup / teardown per query.

Long-running queries that would otherwise hold the lock (like a full tag-count refresh across 250 k photos) are structured as one transaction that reads and returns — no cursor kept open — so the lock is released quickly.

Async model

- `async fn` at the Tauri command boundary is idiomatic. It lets us `await` a background thumbnail generation or a metadata write without blocking a Tokio worker.
- Inside a command, DB operations are synchronous (they take a `MutexGuard`). This is deliberate: SQLite calls are fast enough to keep on the worker thread, and it removes a whole class of cancellation bugs.
- **Rule of thumb:** if a call could block > 5 ms (`std::fs::File::read_to_end` of a JPEG, `spawn_blocking` it. Otherwise, run it inline.

Cancellation and shutdown

- Tauri commands don't get an explicit cancellation token; they run to completion. Long batch operations are structured as a loop over IDs so partial progress is durable even if the user closes the window.
- On close, Tauri drops the main window; pending `spawn_blocking` tasks are given a grace period to finish. Any in-flight metadata write is atomic (write-to-temp + rename inside the source image), so worst case a `.picorg-tmp` file is left behind and cleaned up on the next successful write.

Frontend concurrency

- React Query owns all in-flight IPC promises. It handles deduplication (two components asking for `['image', 5]` share the underlying `invoke` call) and stale-while-revalidate refetches on focus.
- Zustand is synchronous. No middleware, no effects — pure UI state.
- No web workers in v1. The renderer stays responsive because it offloads every non-trivial computation to Rust.

IPC boundary

The contract

The renderer and the Rust core communicate through two mechanisms provided by Tauri:

1. **Commands** — request/response, from renderer to Rust. Each is a Rust `async fn` annotated with `#[tauri::command]` and registered in `lib.rs::invoke_handler`.
2. **Events** — one-way, from Rust to renderer, delivered via `AppHandle::emit(name, payload)` and listened for via `@tauri-apps/api/event::listen`.

Everything is (de)serialised with Serde on the Rust side, JSON on the wire, and TypeScript types on the frontend side. See Tauri command reference for the full list.

Naming conventions

- **Commands** use `snake_case` (Rust) mapped to a same-name string on the frontend: `invoke('update_image_metadata', {…})`.
- **Command arguments** are Rust struct fields in `snake_case`, serialised into the JSON payload as-is. The frontend passes them as `camelCase` object keys because we set `#[serde(rename_all = "camelCase")]` on every argument struct.
- **Events** use a `picorg://` prefix and a resource-style name: `picorg://image-updated`, `picorg://images-deleted`, `picorg://scan`.

Argument struct design

Every command that takes structured data uses a dedicated struct in `src-tauri/src/types.rs`. Optional fields are `Option<T>`, and any field where “unset” and “set to null” must be distinguished uses the custom `double_option` deserializer — an `Option<Option<T>>` where:

Wire form	Deserialised as	Semantics
`{ }` (omitted)	`None`	Don’t touch this field.

Wire form	Deserialised as	Semantics
`{ "field": null }`	`Some(None)`	Clear this field explicitly.
`{ "field": "x" }`	`Some(Some("x"))`	Set to `"x"`.

This distinction matters for the `MetadataPatch` struct: the frontend needs to be able to *clear* a title without also unsetting every other field in the patch.

Type mirroring

`src/types.ts` mirrors the Rust structs. Both sides intentionally avoid derived types (`Omit<T, K>`, `Pick<T, K>`) so a schema change is a single edit in each file.

The frontend types file is a plain declaration file:

```
export interface ImageDetails {
  id: number
  path: string
  filename: string
  // ...
  tags: string[]
  title: string | null
  rating: number | null
  comment: string | null
  metaWrittenAt: number | null
  metaReadAt: number | null
}
```

Every `getImage`, `updateImageMetadata`, etc. wrapper in `ipc.ts` declares its argument and return types, so a Rust command that grows a new field lights up a red squiggle in every caller until the frontend catches up.

Error handling

- Rust commands return `PicOrgResult<T>` (an alias for `Result<T, PicOrgError>`).
- `PicOrgError` is a `thiserror::Error` enum with a `Display` that's safe to show to the user (no internal paths in error messages, etc.).
- On the wire, errors serialise to a plain string. The frontend surfaces them via TanStack Query's `onError` callback and console logs.

Events (Rust → renderer)

Event	Payload	Fired when
`picorg://scan`	`ScanProgress { folder_id, done, total, current }`	During a folder scan.
`picorg://image-updated`	`i64` (image id)	After successful metadata write.
`picorg://images-deleted`	`Vec<i64>` (deleted ids)	After successful delete.

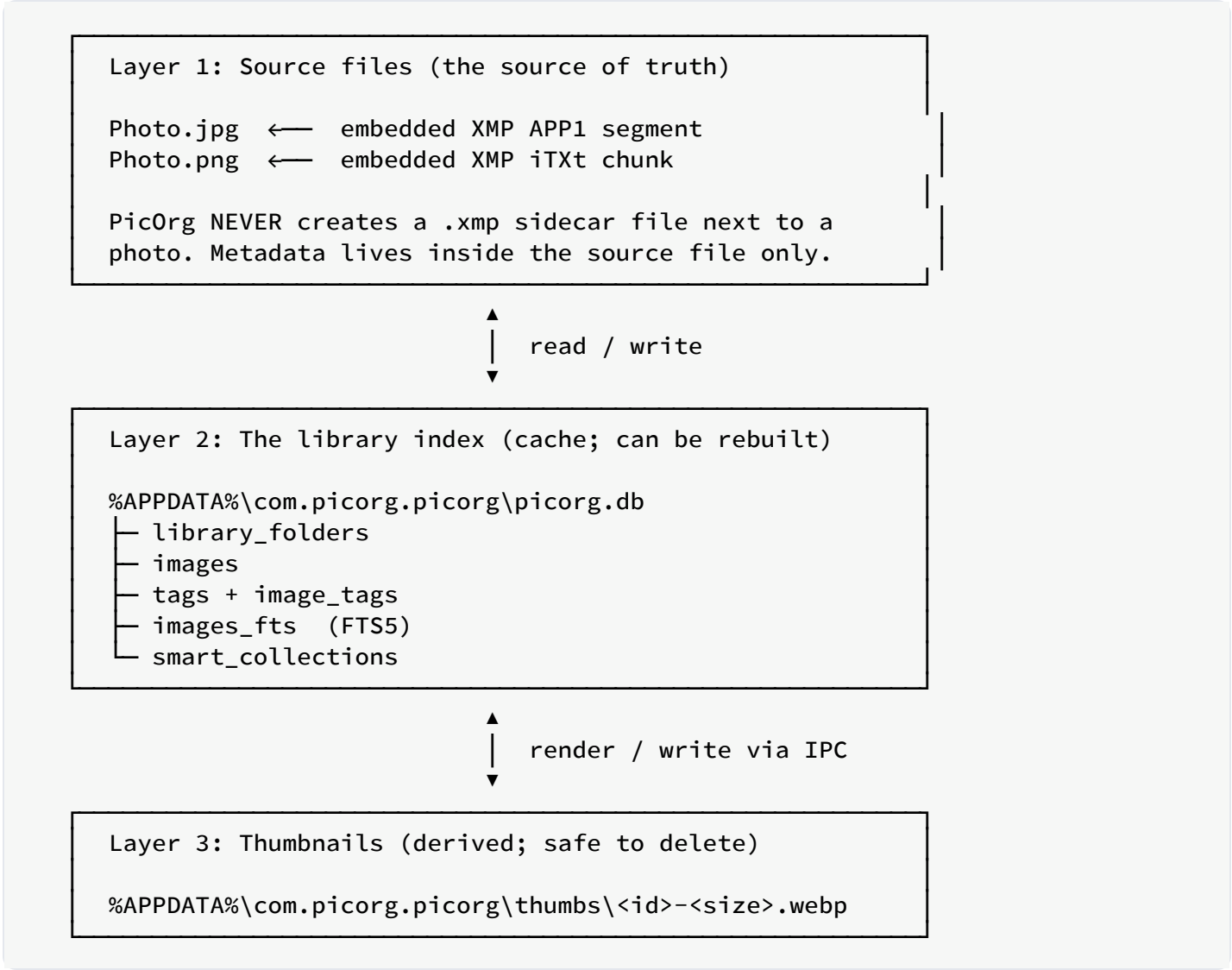
Listeners are attached in `App.tsx` via `useEffect` + `onScanProgress` etc., and each listener updates the appropriate TanStack Query cache or triggers a targeted invalidation.

Capabilities and security

- The renderer has **no direct FS access**. Its `asset:` protocol (used to render thumbnails and full images) is scoped to specific folders declared in `capabilities/default.json`.
- The `tauri.conf.json` `security.csp` blocks inline scripts, remote origins, and `eval`.
- No `dangerouslyUseHttpScheme` or other loosening options are set.
- Commands that operate on paths validate them against the library roots before touching the filesystem.

Data storage architecture

Three layers



Anything a user considers “their data” lives in Layer 1. Layers 2 and 3 exist purely for speed and can be reconstructed at any time by rescanning the library folders.

Layer 1: source of truth (filesystem)

Embedded XMP inside the source file

All user metadata (title, rating, tags, comment) is embedded directly in the image file. Two format-specific writers exist:

- **JPEG (`.jpg`, `.jpeg`, `.jpe`, `.jfif`, `.jif`)** — the XMP packet is injected as an APP1 segment right after SOI (matching Adobe's recommended placement). Any pre-existing standard-XMP or ExtendedXMP APP1 is stripped first, so writes are replace-in- place, not append.
- **PNG (`.png`)** — the XMP packet is stored as an `iTXt` chunk with keyword `XML:com.adobe.xmp`, inserted immediately after `IHDR`. Any existing `XML:com.adobe.xmp` `iTXt` is removed first. CRC-32 is recomputed for the new chunk.

Both rewrites are atomic: temp file next to the original, `sync_all()`, rename. Details in Metadata write path.

The XMP packet uses the same field mapping either way — `dc:title`, `xmp:Rating`, `dc:description`, `dc:subject`, plus a Microsoft-friendly `MicrosoftPhoto:LastKeywordXMP` alias for tags so Windows Explorer's *Tags* column populates.

Unsupported formats

For formats we can't embed into today (HEIC, TIFF, WebP, GIF, BMP, and every camera RAW variant we can scan), `write_metadata_to_source` returns `PicOrgError::MetadataWrite` with a message naming the offending extension. This error is surfaced to the UI so the user sees a clear "can't save" toast — PicOrg does **not** silently fall back to writing a sidecar.

Legacy sidecar files

Older PicOrg versions and other tools (Lightroom, digiKam) may have left `Photo.xmp` files next to some images. PicOrg's reader still picks these up on scan so users don't lose data. On the first successful save into the source file, `write_metadata_to_source` best-effort-deletes the leftover sidecar — after that the photo is the sole source of truth.

Sidecar path detection follows the Lightroom convention: `IMG_1234.jpg → IMG_1234.xmp` (strip original extension). The alternative `IMG_1234.jpg.xmp` (digiKam) form is not read.

Precedence when both exist (read path only)

If both the file's embedded XMP and a legacy sidecar exist, `read_all` merges embedded first and lets the sidecar overwrite, so the sidecar wins. This matches what Lightroom does and means an existing sidecar authored elsewhere still takes effect until PicOrg saves and cleans it up.

Layer 2: the SQLite index

Location: `%APPDATA%\com.picorg.picorg\picorg.db`.

The database has six user tables plus `_migrations` and the FTS5 virtual table `images_fts`. Full schema in Database schema.

Key properties:

- **Single writer** via a `Mutex<Connection>`. All writes are transactional; a failure rolls back cleanly.
- **FTS5 in `contentless_delete=1` mode**. This is a SQLite 3.43+ feature that lets us `DELETE FROM images_fts` before re-inserting a row — necessary because we rebuild an FTS row on every metadata update. See migration `0002_fts_contentless_delete` in `src-tauri/src/db/migrations.rs`.
- **`meta_read_at` / `meta_written_at`** columns on `images` are the pivot for FS synchronisation: if the source file's (or any legacy sidecar's) mtime is newer than `meta_read_at`, PicOrg re- reads the FS before serving a detail request.

Layer 3: thumbnail cache

Location: `%APPDATA%\com.picorg.picorg\thumbs\`.

Format: WebP with quality 80, two sizes per image (`<id>-small.webp`, `<id>-medium.webp`). Small is ~256 px on the long edge, medium ~512 px.

Regeneration:

- On scan, if a thumbnail is missing or the source file's `mtime` is newer than the thumbnail's, PicOrg regenerates.
- On delete, `delete_thumbnails(cache_dir, image_id)` removes every size.
- The user can safely delete the whole `thumbs\` folder; PicOrg regenerates on next request.

Volume assumptions

- Library folders can live on any local volume, including NTFS with long paths (`\\?\`-prefixed) and OneDrive-mounted trees.
- The database and cache always live on `%APPDATA%` — the OS default local storage. Users who want to move it to another volume should create a junction; PicOrg does not have a "move library" UI in v1.

Backup story

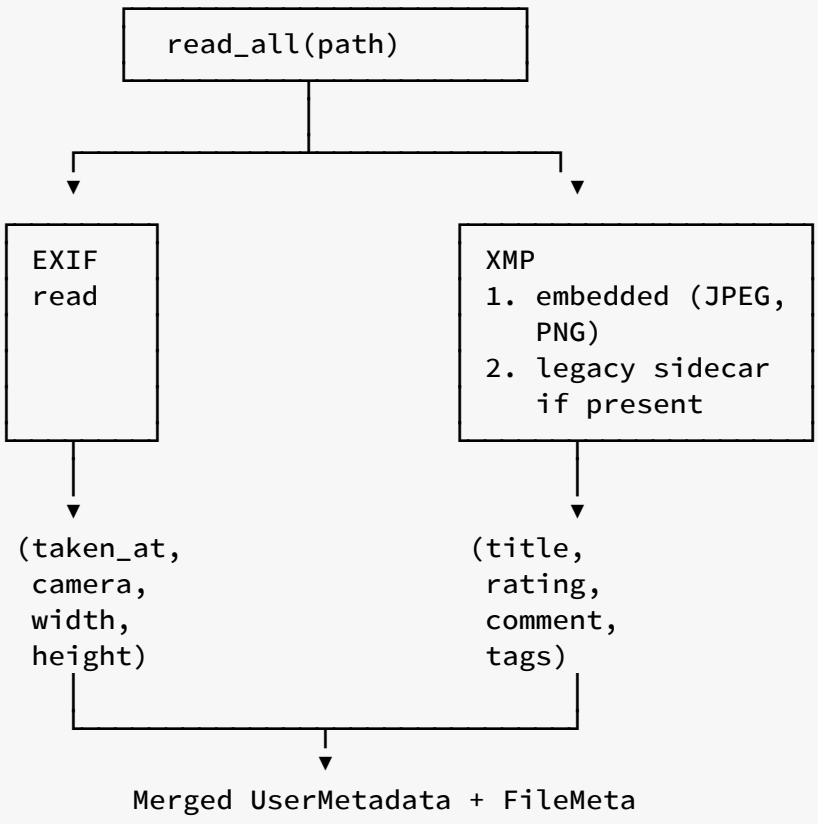
Everything that matters is on the filesystem outside PicOrg's control: your original photos, with the metadata embedded inside them. Back those up and your library is safe. The SQLite database is regenerable and doesn't need to be in a backup — but including it means a restore doesn't require a rescan.

Metadata pipeline

The metadata pipeline is where PicOrg most differs from a plain file browser. It has three sub-pipelines (read, patch, write) and a single invariant that ties them together:

> **Invariant.** After a successful save, the database and the embedded XMP inside the source file reflect the same values. PicOrg does not produce sidecar files; any pre-existing legacy sidecar is removed by the write path after the embed succeeds.

The read pipeline



Ordering within XMP: embedded XMP is parsed first, then any **legacy** sidecar (from an older PicOrg version or from Lightroom) is parsed and its non-empty fields overwrite the embedded ones. This preserves user data during the one-way migration to embed-only. The next successful save embeds the merged state into the source file and removes the sidecar, so this precedence rule matters only until the first edit.

The XMP parser (`quick_xml` state machine in `core/metadata/xmp.rs`) handles both the Adobe standard fields and Microsoft-Explorer variants:

- `dc:title`, `dc:description`, `dc:subject` (Alt/Bag containers)
- `xmp:Rating` (attribute or element)
- `MicrosoftPhoto:LastKeywordXMP` (Windows Explorer's tag store)
- Attribute-only forms (some tools flatten Alt into an attribute)

The patch pipeline

Frontend produces a `MetadataPatch` — a struct where every field is `Option`-wrapped so the caller can express "don't touch" vs. "clear" vs. "set":

```
pub struct MetadataPatch {
    pub title: Option<Option<String>>,
    pub rating: Option<Option<i64>>,
    pub comment: Option<Option<String>>,
    pub tags: Option<Vec<String>>,      // whole list, replaces
    pub tags_add: Option<Vec<String>>,  // additive
    pub tags_remove: Option<Vec<String>>,
}
```

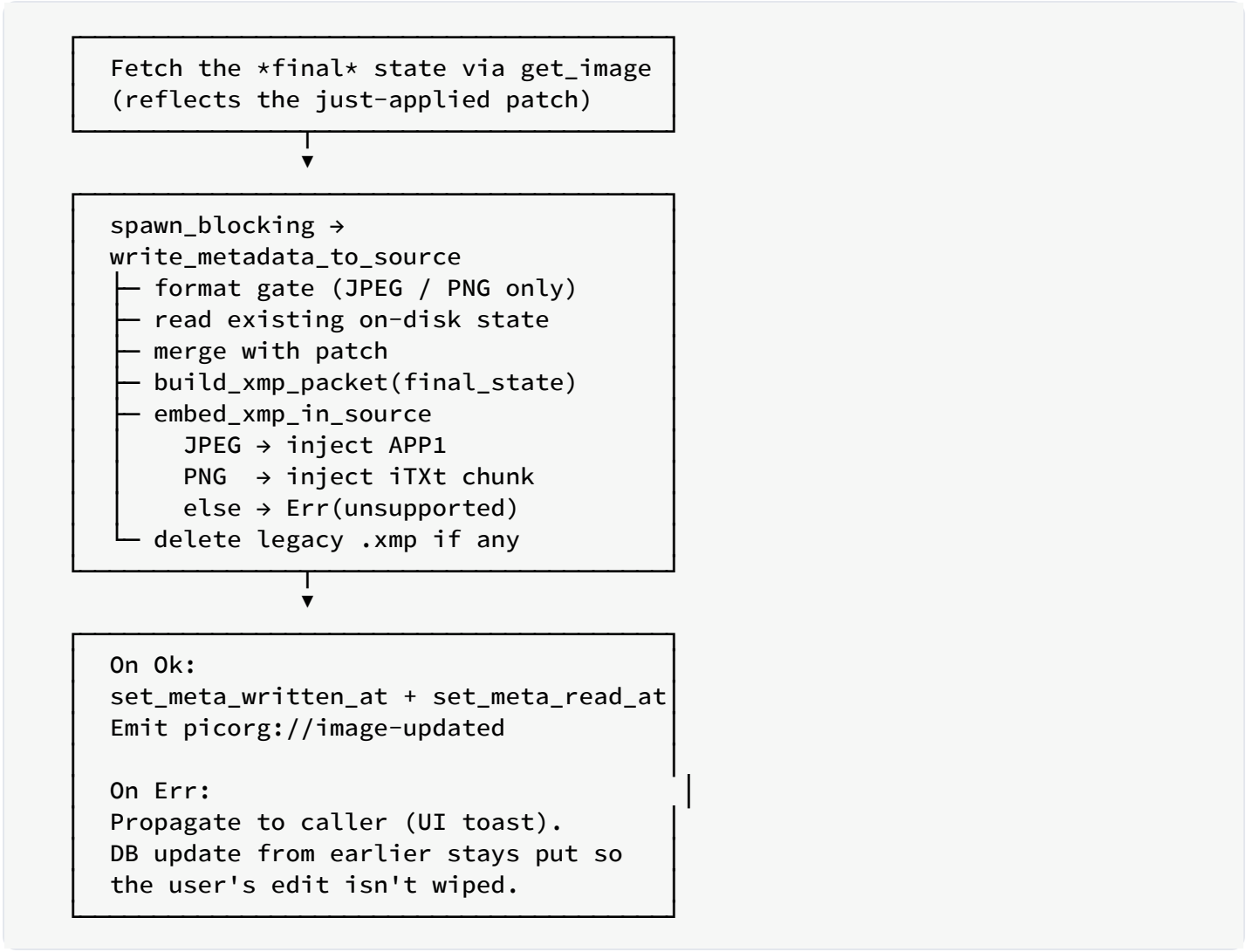
`apply_metadata_patch` (in `db/queries.rs`) runs the whole patch in a single transaction:

1. Update the `images` row (title, rating, comment).
2. If `tags` is set: replace the row's tags.
3. If `tags_add` is set: insert missing.
4. If `tags_remove` is set: delete matching.
5. Rebuild the FTS5 row (DELETE + INSERT).
6. Commit.

If any step fails, the transaction rolls back. Nothing partially lands.

The write pipeline

After `apply_metadata_patch` returns success, the caller (`update_image_metadata` or `batch_update_metadata` via `apply_patch_and_persist`) does:



Three design choices worth calling out:

1. **The XMP packet is built from the DB's final state, not from the patch.** This way, "add tag X" in batch mode writes a packet containing every tag currently on the photo, not just X.
2. **`meta_read_at` is bumped on write** so the FS-refresh check in `get_image` doesn't fire on files PicOrg itself just wrote.
3. **File-write failure surfaces to the UI.** Unlike the old "sidecar is the fallback" design, there's no silent fallback: an unsupported format or a read-only file returns `Err` so the user sees a red toast.

FS refresh on read

Every `get_image` call does:

```
if refresh_needed_from_fs(&path, &cached) {
    let fresh = read_all(&path);
    resync_user_meta_from_fs(&db, id, &fresh);
    set_meta_read_at_now(&db, id);
}
```

`refresh_needed_from_fs` is a simple mtime comparison: if the source file (or any legacy `.xmp` alongside it) was modified after PicOrg's last read, we re-read from disk. This is how a tag added in Windows Explorer becomes visible on next click of that photo. The legacy-sidecar branch remains so that users migrating from Lightroom (or an older PicOrg) still get their existing edits on first scan.

Windows Explorer tag interop specifics

Explorer writes tags into JPEGs using both `dc:subject` (standard) and `MicrosoftPhoto:LastKeywordXMP` (Microsoft-specific). Some older workflows only write the latter, so PicOrg's reader accepts both and unions them, deduping case-insensitively.

On the write side, PicOrg emits `dc:subject` in the XMP packet. Windows Explorer resolves its *Tags* column from either `dc:subject` or `MicrosoftPhoto:LastKeywordXMP`, so a PicOrg- written JPEG shows up correctly in Explorer with only the standard Dublin Core block.

Observability

File-based logging

Every run of PicOrg produces a rolling log file at:

```
%APPDATA%\com.picorg.picorg\logs\picorg.log
```

The logger is initialised in `src-tauri/src/lib.rs::init_logging`, using `tracing-subscriber` with a plain-text formatter and a `RUST_LOG`-compatible env filter (default: `info,picorg_lib=info`).

Format of a typical line:

```
2026-08-17T18:53:12.331332Z  INFO picorg_lib: PicOrg started app_data_dir="…"
2026-08-17T19:22:14.029142Z  INFO picorg_lib::commands::images: get_image:
resynced user metadata from FS id=5
2026-08-18T08:41:03.912483Z  INFO picorg_lib::commands::images:
batch_update_metadata count=3 patch=…
```

- ISO-8601 timestamp with microsecond precision.
- Log level: `TRACE / DEBUG / INFO / WARN / ERROR`.
- Target: usually the module path (`picorg_lib::commands::images`).
- Structured fields on the tail: `id=5`, `count=3`, `?patch`.

Structured fields

We use `tracing`'s field syntax pervasively:

```
tracing::info!(
    id,
    ?patch,                          // Debug-formatted
    op = "update_image_metadata", // static label
    "metadata embedded in source file"
);
```

The `?` prefix invokes `Debug`. Named fields keep the log grep-friendly: to find every batch save, `rg 'batch_update_metadata'` returns just the events, and the id/count/count-succeeded

triplet is on the same line.

Frontend crumbs into the same log

The renderer can push log lines into the Rust log via the `log_frontend` command (`src-tauri/src/commands/diag.rs`). The frontend helper `logFrontend(level, msg)` in `src/ipc.ts` is a best-effort fire-and-forget:

```
logFrontend('info', `applyTags dispatch: ids=${ids.length} add=[…]`)
```

Renderer-side events land in the log with a `target="frontend"` marker, which makes them easy to filter:

```
2026-08-18T08:41:03.900001Z  INFO frontend: applyTags dispatch: ids=3 add=
[vacation] remove=[]
```

This lets us reason about the full IPC round-trip from a single tail of `picorg.log`.

What's logged where

Signal	Level	Emitted by
App started / logging initialised	INFO	`lib.rs::init_logging`, `lib.rs` main run
Migration applied	INFO	`db/migrations.rs::run`
Command entry (`update_image_metadata`, …)	INFO	Each Tauri command that mutates state
`get_image` FS-refresh triggered	INFO	`commands/images.rs::get_image`
Embedded XMP write failed	WARN	`commands/images.rs::apply_patch_and_persist`
Unsupported format for embed	WARN	`core/metadata/write.rs::write_metadata_to_sourc`

Signal	Level	Emitted by
Legacy sidecar cleanup failed	WARN	`core/metadata/write.rs::write_metadata_to_sourc`
Scan errors (unreadable file, permission)	WARN	`core/scanner.rs`
DB migration or startup errors	ERROR	`db/mod.rs`, `lib.rs::run`
Frontend applyTags dispatch	INFO	`features/DetailsPanel.tsx::MultiDetails.applyTa`

Log rotation

The current implementation is a plain append-only file — no rotation. For a v1 release this is fine; the log grows a few kilobytes per session at INFO. A future PR should add `tracing-appender::rolling` with daily rotation or size-based (e.g. 10 MB × 5).

Turning up the volume

Set the `RUST_LOG` environment variable before launching:

```
$env:RUST_LOG = "debug,picorg_lib=trace"
& "$env:LOCALAPPDATA\Programs\picorg\picorg.exe"
```

This will produce a torrent of scanner-level detail, useful when debugging a bad scan on a specific folder.

No telemetry

There is no automatic upload of the log, no crash reporter, no analytics. `picorg.log` is on your disk and stays there unless you explicitly share it (e.g. attach it to a bug report).

Repository layout

```
PicOrg/
├─ src/                                  # React frontend (TypeScript)
│   ├─ App.tsx                           # Root component; layout + event listeners
│   ├─ main.tsx                          # ReactDOM entry
│   ├─ index.css                         # Tailwind directives + global styles
│   ├─ ipc.ts                            # Typed Tauri command wrappers
│   ├─ store.ts                          # Zustand global UI state
│   ├─ types.ts                          # Mirrors Rust types
│   └─ features/
│      ├─ TopBar.tsx                     # Top-of-window: add folder, rescan,
search, sort
│      ├─ Sidebar.tsx                    # Left: library / rating / tag filters
│      ├─ ImageGrid.tsx                  # Virtualised photo grid
│      ├─ DetailsPanel.tsx               # Right: SingleDetails / MultiDetails
│      ├─ StarRating.tsx                 # 5-star clickable widget
│      ├─ TagInput.tsx                   # Tag entry with autocompletion
│      ├─ Thumbnail.tsx                  # <img> wrapper with async src
│      └─ StatusBar.tsx                  # Bottom: scan progress, app version
│
├─ src-tauri/                            # Rust backend (Cargo package)
│   ├─ Cargo.toml
│   ├─ tauri.conf.json                   # App config: identifier, security, window
│   ├─ build.rs
│   ├─ capabilities/
│   │  └─ default.json                   # Capability manifest (asset scopes, etc.)
│   ├─ examples/
│   │  ├─ dump_meta.rs                   # Diagnostic: dump everything about a photo
│   │  └─ dump_tag_usage.rs              # Diagnostic: find photos tagged X
│   ├─ tests/
│   │  └─ metadata_fs.rs                 # Integration tests for metadata pipeline
│   └─ src/
│      ├─ main.rs                       # Windows subsystem entry — calls lib::run
│      ├─ lib.rs                        # Tauri builder, logging, handler
registration
│      ├─ error.rs                      # PicOrgError, PicOrgResult
│      ├─ types.rs                      # Rust-side IPC types (mirror src/types.ts)
│      ├─ db/
│      │  ├─ mod.rs                     # Db (Mutex<Connection>) + with_conn helper
│      │  ├─ migrations.rs              # SQL migrations (schema + FTS fix)
│      │  └─ queries.rs                 # apply_metadata_patch, get_image, etc.
│      ├─ core/
│      │  ├─ mod.rs                     # AppServices, image-ext filter, dirs
│      │  ├─ scanner.rs                 # Parallel folder scan
│      │  ├─ thumbnail.rs               # Thumb generation + caching
│      │  └─ metadata/
│      │     ├─ mod.rs
│      │     ├─ read.rs                 # EXIF + embedded XMP + legacy sidecar read
│      │     ├─ write.rs                # Embed-into-source writer
│      │     ├─ sidecar.rs              # Legacy `.xmp` path helper (read +
cleanup)
```

```
│       │   └── xmp.rs                  # XMP parse + build + embed in JPEG / PNG
│       └── commands/                   # Tauri command handlers
│           ├── mod.rs
│           ├── library.rs              # add/remove/list folders, rescan
│           ├── images.rs               # query, get, update, batch_update, delete
│           ├── tags.rs                 # list / rename / delete tags
│           ├── collections.rs          # smart collections (skeleton in v1)
│           ├── thumbs.rs               # thumb + asset path resolvers
│           └── diag.rs                 # log_frontend bridge
│
├── scripts/
│   ├── screenshot-app.ps1              # Verify UI screenshot
│   ├── screenshot-scrolled.ps1         # Screenshot with programmatic scroll
│   └── test-multiselect-tag.ps1        # E2E UI test skeleton
│
├── docs/                               # This documentation
│   ├── user-manual/                    # mdBook: user-facing manual
│   ├── developer/                      # mdBook: this guide
│   ├── shared/                         # Shared CSS / JS for both books
│   ├── index.html                      # Landing page linking both docs
│   └── build.ps1                       # HTML + PDF build script
│
├── index.html                          # Vite entry
├── vite.config.ts
├── tailwind.config.js
├── postcss.config.js
├── tsconfig.json
├── package.json
├── README.md
└── .gitignore
```

What lives where

- **Anything user-visible** (button text, layout, animations) is under `src/`.
- **Anything touching disk** (files, DB) is under `src-tauri/src/`. If you find frontend code doing FS work, that's a bug.
- **Anything that runs at build time** (Tauri config, capabilities, Cargo settings) is under `src-tauri/` outside `src/`.
- **Documentation source** is under `docs/`. Generated HTML lands in `docs/user-manual/book/` and `docs/developer/book/`; PDFs land in `docs/`.

Backend modules

`lib.rs` — app entry

Responsibilities:

- Initialise file-based logging (`init_logging`).
- Build `AppServices { db, cache_dir, … }` under `Arc`.
- Register every Tauri plugin (`dialog`, `opener`) and command handler in `invoke_handler!`.
- Manage the app run loop.

Notable pitfalls:

- `init_logging` **must not** call `tracing_subscriber::fmt().init()` if a Tauri plugin (like the removed `tauri-plugin-log`) already set a global subscriber. The prior implementation crashed with `PluginInitialization("log", "attempted to set a logger after…")`; the current version owns the logger outright.

`types.rs` — IPC types

- `ImageSummary`, `ImageDetails` — full and abridged views of an image row.
- `MetadataPatch` — the "what to change" payload for `update_image_metadata` and `batch_update_metadata`.
- `double_option` module — custom Serde deserializer for `Option<Option<T>>`. Distinguishes "field missing" from "field explicitly null" on the wire.

Every struct uses `#[serde(rename_all = "camelCase")]` so JSON keys are camelCase but Rust fields stay snake_case.

`error.rs`

`PicOrgError` enum (via `thiserror`):

- `Io(std::io::Error)` — filesystem errors.

- `Db(rusqlite::Error)` — DB errors.
- `MetadataRead(String)`, `MetadataWrite(String)` — user-facing metadata failures.
- `Scan(String)` — scanner-specific issues.
- `Internal(String)` — anything else.

All commands return `PicOrgResult<T>` (= `Result<T, PicOrgError>`). The `Display` impl for `PicOrgError` is safe to surface to the user (no absolute paths in strings without context).

db/mod.rs

Thin wrapper around `rusqlite::Connection`:

```
pub struct Db {
    conn: Arc<Mutex<Connection>>,
}

impl Db {
    pub fn open(path: &Path) -> PicOrgResult<Self> { … }

    pub fn with_conn<F, T>(&self, f: F) -> PicOrgResult<T>
    where
        F: FnOnce(&Connection) -> PicOrgResult<T>,
    { … }
}
```

The single-mutex approach keeps SQLite calls simple and avoids connection pooling. WAL mode is enabled on open, so concurrent readers don't stall writers.

db/migrations.rs

Two migrations in the ordered `MIGRATIONS` slice:

1. `0001_init` — full schema (see Database schema).
2. `0002_fts_contentless_delete` — recreate `images_fts` with `contentless_delete=1` and blank out `meta_read_at` so the FS sync path re-runs for every image.

Migrations run at startup inside `run(conn)`. Each is applied atomically and recorded in the `_migrations` table.

db/queries.rs

The functional heart of the DB layer. Highlights:

Function	Purpose
`apply_metadata_patch(db, id, p)`	Apply a `MetadataPatch` to an image in a single txn.
`get_image(db, id)`	Fetch full `ImageDetails` including tags.
`query_images(db, filter, sort, page)`	Paginated grid query with sidebar filters + FTS.
`list_tags(db, prefix)`	Tags with counts, optionally prefix-filtered.
`rename_tag(db, old, new)`	Global rename in one txn (including FTS rebuild).
`delete_tag(db, name)`	Global delete of a tag.
`get_image_paths(db, ids)`	Bulk fetch of paths for delete.
`delete_image_rows(db, ids)`	Bulk row deletion after files are gone.
`resync_user_meta_from_fs(db, id, meta)`	Sync FS-read metadata into DB, not touching
	scan-derived fields like size/mtime.
`set_meta_read_at_now(db, id)`	Timestamp helper.
`set_meta_written_at(db, id, t)`	Timestamp helper.

Every mutation goes through a `Transaction` obtained via `conn.transaction()?`, and every one calls `rebuild_fts_row_tx` at the end to keep the search index in sync.

core/scanner.rs

Pipeline for `add_library_folder` and `rescan_*`:

1. **Walk.** `jwalk::WalkDir` traverses the folder in parallel.
2. **Filter.** Extensions checked against `IMAGE_EXTS` (see `core/mod.rs`).
3. **Diff.** For each file, look up by path in `images`; skip if `mtime_ms` matches.

4. **Extract.** In parallel via `rayon`: EXIF read, XMP read, content hash (`XXH3`).
5. **Upsert.** DB writes are serialised via the single connection mutex; scanner buffers batches to amortise txn overhead.
6. **Thumbnails.** Enqueue small + medium.
7. **Progress.** Emit `picorg://scan { done, total, current }`.

Scanning is bounded by disk read speed on cold cache and by CPU on warm cache.

`core/thumbnail.rs`

```
pub fn ensure_thumbnails(cache_dir: &Path, src: &Path, image_id: i64) ->
Result<()>;
pub fn thumb_path(cache_dir: &Path, image_id: i64, size: ThumbSize) -> PathBuf;
pub fn delete_thumbnails(cache_dir: &Path, image_id: i64);
```

Decode via `image::open`, resize via `fast_image_resize` (SIMD Lanczos3), encode via `webp::Encoder` at quality 80.

`core/metadata/read.rs`

`read_all(path) -> ImageMeta` runs the read pipeline:

1. `exif::Reader::read_from_container` for taken time + camera fields.
2. `xmp::extract_embedded_xmp(path)` for embedded XMP (JPEG APP1 or PNG iTXt).
3. Read any legacy `<image>.xmp` sidecar for backward compatibility.
4. Merge sidecar-over-embedded via `apply_user_meta` (sidecar wins for user metadata so a Lightroom-authored `.xmp` still takes effect on first scan).

Non-fatal errors are collected into a per-file warning log; the scanner continues on the next file.

`core/metadata/xmp.rs`

Everything XMP:

- `extract_embedded_xmp(path)` — walker for JPEG APP1 and PNG iTXt XMP chunks.
- `parse_user_metadata(bytes)` — streaming `quick_xml` parser that extracts the title / description / rating / subjects / MSFT keywords we care about.

- `build_xmp_packet(&UserMetadata)` — writes a standard XMP packet with `dc:title`, `dc:description`, `xmp:Rating`, and `dc:subject`. Windows Explorer's *Tags* column resolves from either `dc:subject` or `MicrosoftPhoto:LastKeywordXMP`, so PicOrg only emits the standard Dublin Core form.
- `embed_xmp_in_source(path, packet_bytes)` — dispatches to `embed_xmp_in_jpeg` (APP1) or `embed_xmp_in_png` (iTXt). Returns `Ok(false)` for unsupported extensions so the caller can produce a clear format-aware error.
- `format_supports_embedded_xmp(ext)` — single source of truth for which extensions the writer supports (currently JPEG + PNG).

`core/metadata/write.rs`

`write_metadata_to_source` is the single entry point for saves:

1. **Format gate.** Unsupported extensions return `Err(_)` before any I/O.
2. Read the *current* on-disk metadata (so we don't drop fields we don't touch).
3. Apply the patch fields (title / description / rating / subjects).
4. `embed_xmp_in_source` — atomic temp+rename inside the source file (JPEG APP1 or PNG iTXt).
5. Best-effort delete of any leftover `<image>.xmp` from a previous PicOrg version or from Lightroom.

Failure semantics:

- Unsupported format = `Err(_)` before any disk touch.
- Embed failure (read-only, disk full, corrupt file) = `Err(_)`.
- Sidecar cleanup failure = logged as WARN, save still returns `Ok(())` because the source file already carries the metadata.

`commands/*.rs`

Each file exposes 3–5 `#[tauri::command]` async functions. See Tauri command reference for the full list.

Common patterns:

- `services: State<'_, Arc<AppServices>>` for shared state.
- `app_handle: AppHandle` for emitting events.
- Return `PicOrgResult<T>`; Serde does the JSON legwork.

- Tracing spans on entry so `picorg.log` shows every IPC hit.

Frontend modules

The frontend is intentionally small — the goal is to be a translation layer between UI events and Tauri commands, not to hold domain logic.

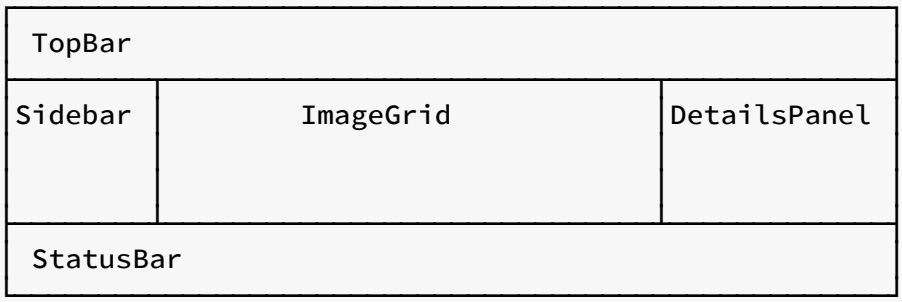
`src/main.tsx`

Boots React. Wraps `<App />` in a `QueryClientProvider` with a single QueryClient configured for:

- `staleTime: 0` — refetch on refocus.
- `retry: 1` — one retry on network / IPC errors.
- Query cache keys convention (below).

`src/App.tsx`

Owns the top-level layout:



Also hosts the global keyboard listener (Delete key), and mounts event listeners for `picorg://image-updated` and `picorg://images-deleted` — both of which invalidate the relevant query keys.

`src/store.ts` (Zustand)

Global UI-only state:

```
interface Store {
  view: View                // 'all' | { folderId } | { rating } | { tag }
  search: string
  sort: ImageSort
  extraFilter: ImageFilter  // sidebar filters composed
  selection: { ids: Set<number>; anchor: number | null }
  detailsOpen: boolean
  // setters
  setView, setSearch, setSort, setSelection, clearSelection, …
}
```

No persistence in v1 — reloading the app resets to `view: 'all'`, empty search, no selection.

`filterFromView(view, extra, search)` composes the sidebar view, the extra filter, and the search string into a single `ImageFilter` sent to `query_images`.

src/ipc.ts

Thin, typed wrappers around `invoke`:

```
export const updateImageMetadata = (id: number, patch: MetadataPatch) =>
  invoke<ImageDetails>('update_image_metadata', { id, patch })

export const batchUpdateMetadata = (ids: number[], patch: MetadataPatch) =>
  invoke<number[]>('batch_update_metadata', { ids, patch })

// …
```

Plus event-listener helpers:

```
export const onImageUpdated = (h: (id: number) => void) =>
  listen<number>('picorg://image-updated', e => h(e.payload))
```

And the diagnostic helper `logFrontend(level, msg)` for pushing crumbs into the backend log.

src/types.ts

Every IPC struct duplicated from Rust, kept in sync manually. See IPC boundary for the rationale (no derived types).

src/features/

TopBar.tsx

- Add folder button (opens Tauri dialog, calls `add_library_folder`, invalidates `['folders']` and `['images']`).
- Rescan button (calls `rescan_all`).
- Live search input (debounced 200 ms → `useStore.setSearch`).
- Sort dropdown + direction toggle.
- Toggle-details icon.

Sidebar.tsx

- `['folders']`, `['tags']` queries fed to three collapsible sections.
- Click handlers set `useStore.view` and clear other filters.
- Right-click context menu for `Remove folder`, `Rescan folder`, `Rename tag`, `Delete tag`.

ImageGrid.tsx

- `useQuery(['images', filter, sort, page])` returns paginated results.
- `useVirtualizer` from TanStack Virtual computes visible rows.
- Each tile is a `Thumbnail` component; `Ctrl / Shift / plain` click logic mutates `useStore.selection`.

DetailsPanel.tsx

- Reads `useStore.selection` and dispatches:
  - 0 selected → placeholder.
  - 1 selected → `<SingleDetails id=… />`.
  - 1 selected → `<MultiDetails ids=… />`.

`SingleDetails` seeds local edit state from the query result only when the `id` changes (guarded by `lastLoadedId.current`). This avoids the “stomp typing on refetch” bug that used to happen.

`MultiDetails` uses refs (`tagsAddRef`, `tagsRemoveRef`) mirroring state so the mutation dispatch always sees the latest values, even if a blur-triggered `setTagsAdd` and a click on `Apply` land in the same React batch.

`StarRating.tsx`

Purely presentational — takes `value: number | null` and `onChange`, renders five buttons. Clicking the current value sends `null` (reset).

`TagInput.tsx`

- Controlled by parent's `tags: string[]` + `onChange(next: string[])`.
- Commits the current draft on Enter, Space, Comma, or blur.
- Autocompletion pulled from `list_tags(prefix)` via TanStack Query.
- Backspace on empty input pops the last tag.

`Thumbnail.tsx`

- Resolves `getThumbPath(id, 'small')` on mount, sets `<img src>` to the returned asset URL.
- Placeholder while the thumbnail is being generated (rare, only on first scan).

`StatusBar.tsx`

- Listens to `picorg://scan` events, shows a live progress bar and message.
- Static labels for app version, DB size, thumbnail cache size.

Query key conventions

Query key	Fetched by	Invalidated by
`['folders']`	`listLibraryFolders`	add/remove/rescan folder, delete images
`['tags']`	`listTags`	any metadata update, tag rename/delete

Query key	Fetched by	Invalidated by
`['tag', prefix]`	`listTags(prefix)`	any metadata update
`['images', filter, sort, page]`	`queryImages`	any image mutation
`['image', id]`	`getImage`	`update_image_metadata` (via `setQueryData`),
		`batch_update_metadata` (via `invalidateQueries`),
		`picorg://image-updated` event
`['smartCollections']`	`listSmartCollections`	create/delete collection

Styling

- Tailwind for utility classes.
- Global styles in `src/index.css`: dark background, rounded buttons, focus rings.
- No CSS-in-JS. Component-scoped styles are className concatenations via `clsx` when needed.

Database schema

The database is a single SQLite file at `%APPDATA%\com.picorg.picorg\picorg.db`. Schema definition lives in `src-tauri/src/db/migrations.rs`.

Tables

`library_folders`

Every folder the user has added to the library.

Column	Type	Notes
id	INTEGER	PK, autoincrement.
path	TEXT	Absolute, canonical, UNIQUE.
added_at	INTEGER	Unix ms.
last_scan_at	INTEGER	Unix ms; NULL before first scan finishes.

`images`

One row per image file found by the scanner.

Column	Type	Notes
id	INTEGER	PK, autoincrement.
folder_id	INTEGER	FK → `library_folders(id)` ON DELETE CASCADE.
path	TEXT	Absolute path, UNIQUE.
filename	TEXT	Basename (`IMG_1234.jpg`).
ext	TEXT	Lowercase extension, no dot (`jpg` , `heic` , …).
size_bytes	INTEGER	From `fs::Metadata::len()` .
mtime_ms	INTEGER	Modification time in Unix ms.

Column	Type	Notes
width	INTEGER	Nullable; from EXIF or image decode.
height	INTEGER	Nullable.
content_hash	TEXT	Nullable; XXH3 128-bit hex digest.
taken_at	INTEGER	Nullable; Unix ms from EXIF `DateTimeOriginal`.
camera_make	TEXT	Nullable.
camera_model	TEXT	Nullable.
title	TEXT	Nullable; user-editable via UI.
rating	INTEGER	CHECK: NULL or 0..=5.
comment	TEXT	Nullable.
meta_written_at	INTEGER	Nullable; last time PicOrg wrote XMP for this photo.
meta_read_at	INTEGER	Nullable; last time PicOrg read XMP from disk into DB.
missing	INTEGER	0/1 flag for “file expected but not found” (unused v1).

Indexes:

- `idx_images_folder` (folder_id)
- `idx_images_taken_at` (taken_at)
- `idx_images_rating` (rating)
- `idx_images_filename` (filename)

`tags`

The tag vocabulary.

Column	Type	Notes
id	INTEGER	PK, autoincrement.
name	TEXT	UNIQUE COLLATE NOCASE.

`image_tags`

Many-to-many join between `images` and `tags`.

Column	Type	Notes
image_id	INTEGER	FK → `images(id)` ON DELETE CASCADE.
tag_id	INTEGER	FK → `tags(id)` ON DELETE CASCADE.
PK		(image_id, tag_id).

Indexes:

- `idx_image_tags_tag` (tag_id) — accelerates "photos with tag X".

`images_fts` (virtual, FTS5)

Full-text search index over four columns.

```
CREATE VIRTUAL TABLE images_fts USING fts5(
    title, comment, filename, tags,
    content='',
    contentless_delete=1,
    tokenize='unicode61 remove_diacritics 2'
);
```

Key attributes:

- **Contentless** (`content=''`): the FTS table doesn't store the original text; queries return `rowid` (= `images.id`) and we join.
- **`contentless_delete=1`** (SQLite 3.43+): lets us `DELETE FROM images_fts` before re-inserting a row. Without it, `rebuild_fts_row_tx` fails and rolls back every metadata write. See migration `0002_fts_contentless_delete`.
- **Diacritics removed at level 2**: `naive` and `naïve` are the same token.

`smart_collections`

Skeleton in v1 — data model exists, UI to come.

Column	Type	Notes
id	INTEGER	PK, autoincrement.

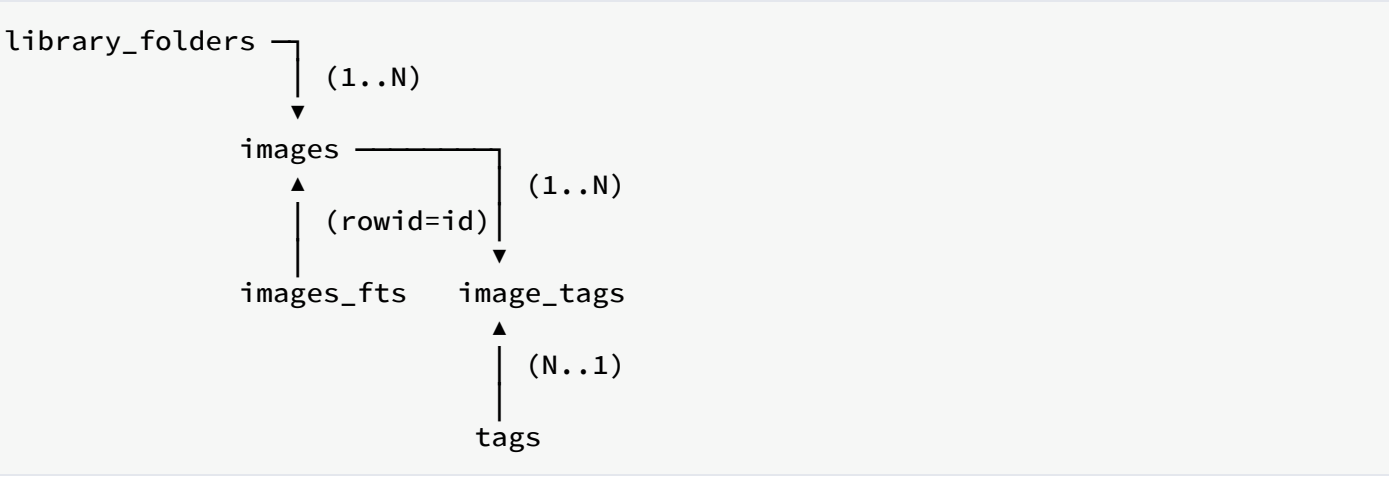
Column	Type	Notes
name	TEXT	
filter	TEXT	JSON-serialised `ImageFilter`.
sort_order	INTEGER	

`_migrations`

Housekeeping.

Column	Type	Notes
name	TEXT	PK; migration name (e.g. `0001_init`).
applied_at	INTEGER	Unix ms when the migration ran.

Relationships



Migrations

Name	Effect
`0001_init`	Full schema above (without `contentless_delete`).
`0002_fts_contentless_delete`	Drop + recreate `images_fts` with `contentless_delete=1`;

Name	Effect
	repopulate from live data; blank `meta_read_at` to force
	FS re-read.

Each migration is applied atomically inside a transaction and recorded in `_migrations` on success.

Tauri command reference

Every command below is registered in `src-tauri/src/lib.rs` inside `tauri::generate_handler![…]`. All are `async fn`, return `PicOrgResult<T>`, and use camelCase JSON on the wire.

Library management

`add_library_folder`

```
async fn add_library_folder(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    path: String,
) -> PicOrgResult<LibraryFolder>;
```

Add a folder to the library and start a background scan. Emits `picorg://scan` progress events. Returns the created row.

`remove_library_folder`

```
async fn remove_library_folder(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> PicOrgResult<()>;
```

Removes the folder row (CASCADE removes every image and image_tags row). Files on disk are untouched.

`list_library_folders`

```
async fn list_library_folders(
    services: State<'_, Arc<AppServices>>,
) -> PicOrgResult<Vec<LibraryFolder>>;
```

`rescan_folder`

```
async fn rescan_folder(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    id: i64,
) -> PicOrgResult<ScanResult>;
```

Re-walks the specified folder. Incremental: only files with a newer `mtime` than what's in the DB get re-processed.

`rescan_all`

```
async fn rescan_all(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
) -> PicOrgResult<Vec<ScanResult>>;
```

Images

`query_images`

```
async fn query_images(
    services: State<'_, Arc<AppServices>>,
    filter: Option<ImageFilter>,
    sort: Option<ImageSort>,
    page: Option<Pagination>,
) -> PicOrgResult<Page<ImageSummary>>;
```

Paginated grid query. `filter` composes:

- `folder_id: Option<i64>`
- `min_rating: Option<i64>`
- `tag: Option<String>`
- `search: Option<String>` — passed through FTS5.

Returns `Page { items, total, page, page_size }`.

`get_image`

```
async fn get_image(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> PicOrgResult<ImageDetails>;
```

Returns full details for one image. **Side effect:** if the source file's mtime (or any legacy `.xmp` sidecar's mtime) is newer than `meta_read_at`, re-reads metadata from disk and updates the DB *before* returning.

`update_image_metadata`

```
async fn update_image_metadata(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    id: i64,
    patch: MetadataPatch,
) -> PicOrgResult<ImageDetails>;
```

Apply a metadata patch, embed the merged XMP into the source file, delete any legacy `.xmp` sidecar, and return the new state. Emits `picorg://image-updated`. Returns `Err` on unsupported formats (anything other than JPEG or PNG) or file-write failures.

`batch_update_metadata`

```
async fn batch_update_metadata(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    ids: Vec<i64>,
    patch: MetadataPatch,
) -> PicOrgResult<Vec<i64>>;
```

Same as `update_image_metadata` but for many photos. Returns the list of IDs that succeeded; failures are logged and skipped.

`delete_images`

```
async fn delete_images(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    ids: Vec<i64>,
    permanent: Option<bool>,
) -> PicOrgResult<DeleteResult>;
```

Move to Recycle Bin (default) or permanently delete. Also removes any legacy `.xmp` sidecar + thumbnails + DB row. Returns per-file success/failure.

Tags

`list_tags`

```
async fn list_tags(
    services: State<'_, Arc<AppServices>>,
    prefix: Option<String>,
) -> PicOrgResult<Vec<TagStats>>;
```

Every tag with a photo count. Optional prefix filter for autocompletion.

`rename_tag`

```
async fn rename_tag(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    old_name: String,
    new_name: String,
) -> PicOrgResult<()>;
```

Global rename across every photo. Rewrites every affected embedded XMP packet in the source files.

`delete_tag`

```
async fn delete_tag(
    services: State<'_, Arc<AppServices>>,
    app_handle: AppHandle,
    name: String,
) -> PicOrgResult<()>;
```

Remove a tag from every photo. Files are updated the same way as rename.

Smart collections (skeleton)

`list_smart_collections`

```
async fn list_smart_collections(
    services: State<'_, Arc<AppServices>>,
) -> PicOrgResult<Vec<SmartCollection>>;
```

`create_smart_collection`

```
async fn create_smart_collection(
    services: State<'_, Arc<AppServices>>,
    name: String,
    filter: ImageFilter,
) -> PicOrgResult<SmartCollection>;
```

`delete_smart_collection`

```
async fn delete_smart_collection(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> PicOrgResult<()>;
```

Thumbnails and image paths

`get_thumb_path`

```
async fn get_thumb_path(
    services: State<'_, Arc<AppServices>>,
    id: i64,
    size: ThumbSize,
) -> PicOrgResult<String>;
```

Returns the absolute path of a cached thumbnail; generates it on demand if missing.

`get_image_path`

```
async fn get_image_path(
    services: State<'_, Arc<AppServices>>,
    id: i64,
) -> PicOrgResult<String>;
```

Returns the absolute path of the source image (used by the details panel to render a large preview via `convertFileSrc`).

Diagnostics

`log_frontend`

```
fn log_frontend(level: String, msg: String) -> PicOrgResult<()>;
```

Push a log line into `picorg.log` from the renderer. `level` is one of `debug|info|warn|error`. Emitted with `target="frontend"` for easy filtering.

Scanner algorithm

Objective

Given a library folder path, populate the `images` table with a row per supported image file, extract metadata, generate thumbnails, and keep the DB in sync with the filesystem on subsequent runs — all while staying responsive.

Pipeline

```
walk()  →  filter()  →  diff()  →  extract()  →  upsert()  →  thumbnail()
   │          │           │           │            │             │
jwalk::   ext-in-      mtime        rayon         DB          thumb
WalkDir   IMAGE_EXTS   match?      parallel    serialised    pipeline
 par      ↳ skip       ↳ skip                   (mutex)
 walk    others        else read
```

Progress events (`picorg://scan { folder_id, done, total, current }`) are emitted every N files (default 20).

Stage 1 — walk

`jwalk::WalkDir::new(root).parallelism(Parallelism::RayonNewPool(n))` walks the folder tree in parallel across all cores, streaming `DirEntry` values. Symlinks are followed (with cycle detection); hidden files are respected per OS convention.

Stage 2 — filter

An entry is kept iff:

- It's a regular file (not a directory, socket, or device).
- Its lowercase extension is in `core::IMAGE_EXTS` : `jpg jpeg jpe jfif jif png gif bmp tif tiff webp heic heif` `cr2 cr3 nef arw dng raf orf pef x3f` .

- Its filename doesn't start with `.` (dotfiles).

Discarded entries increment a counter used for the progress denominator but otherwise do no work.

Stage 3 — diff

For each kept entry, PicOrg looks up the path in `images`:

```
SELECT id, mtime_ms FROM images WHERE path = ?1
```

- **New file** (no row): full extract required.
- **Existing, mtime unchanged**: skip.
- **Existing, mtime changed**: partial re-extract (metadata + hash), leaving other columns intact.

The lookup is O(log n) thanks to the UNIQUE index on `images.path`.

Stage 4 — extract

For each file that needs work, the following runs on a rayon worker:

1. **Stat** — `fs::metadata(path)` for `size_bytes` and `mtime_ms`.
2. **Content hash** — stream the file through `XXH3_128`. On an SSD this is disk-bound at ~2 GB/s; on HDD it's the bottleneck.
3. **EXIF** — `kamadak-exif::Reader::read_from_container` for `taken_at`, `camera_make`, `camera_model`, `width`, `height`.
4. **XMP** — `metadata::read::read_all(path)` for `title`, `rating`, `comment`, `tags`.

Any per-file failure is logged (`WARN`) and doesn't abort the folder scan. The file gets a row with whatever metadata succeeded; missing fields are left NULL.

Stage 5 — upsert

The DB is the single writer. To amortise transaction overhead, extracted rows are pushed to a bounded channel; a single "writer task" drains the channel and commits batches of up to 128 rows in one transaction:

```
let tx = conn.transaction()?;
for row in batch { upsert_image(&tx, &row)?; }
tx.commit()?;
```

The `upsert_image` SQL is:

```
INSERT INTO images (path, folder_id, filename, ext, size_bytes, mtime_ms,
                    width, height, content_hash, taken_at,
                    camera_make, camera_model, title, rating, comment,
                    meta_read_at)
VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
ON CONFLICT(path) DO UPDATE SET
    size_bytes    = excluded.size_bytes,
    mtime_ms      = excluded.mtime_ms,
    width         = excluded.width,
    height        = excluded.height,
    content_hash  = excluded.content_hash,
    taken_at      = excluded.taken_at,
    camera_make   = excluded.camera_make,
    camera_model  = excluded.camera_model,
    title         = excluded.title,
    rating        = excluded.rating,
    comment       = excluded.comment,
    meta_read_at  = excluded.meta_read_at;
```

After the row lands, tags are reconciled: everything in `image_tags` for this image is deleted, then re-inserted from the fresh tag list. Both happen in the same transaction as the image upsert.

The FTS5 row is rebuilt via `rebuild_fts_row_tx` at the very end.

Stage 6 — thumbnails

Once a row is in the DB, the scanner enqueues a thumbnail task for that image id. Thumbnails run on the same rayon pool but with lower priority so they don't starve extraction.

`ensure_thumbnails(cache_dir, src_path, image_id)`:

1. For each size (small, medium):
   - Path = `cache_dir / "<id>-<size>.webp"`.
   - Skip if the thumbnail's mtime ≥ the source's mtime.
2. Decode the source with `image::open`.
3. Resize with `fast_image_resize::Resizer` (Lanczos3, SIMD).
4. Encode with `webp::Encoder::new(&image).encode(80)`.

5. Write bytes atomically (temp + rename).

Failures at any stage are logged and don't affect the DB row.

Handling deletions

If a file that was in the DB is no longer found on disk during a scan, the scanner does **not** delete the row automatically — the `missing` flag in the `images` table exists to mark it (v1: not exposed in UI). A future contribution can add an explicit "Clean up missing photos" UI action.

Incremental performance

The diff step is what makes rescans fast:

- 50 000 photos, no changes: `~2 seconds` (mostly the DB lookup and a stat call per file).
- 50 000 photos, 100 new: `~5 seconds` (mostly hashing + XMP read for the 100).
- 50 000 photos, cold cache (thumbnails missing): `~2 minutes` (bounded by encoding).

Thumbnail pipeline

Sizes

Two cached sizes per image, both stored as WebP quality 80:

Enum name	Long-edge px	Used by
`Small`	~256	`ImageGrid` tiles (main use case).
`Medium`	~512	(Reserved for a future zoom-in mode.)
`Large`	~1024	(Currently only generated on request.)

Aspect ratio is preserved — the short edge is proportional. A 6000×4000 source becomes a 256×170 small and 512×341 medium.

Location

Files live in `%APPDATA%\com.picorg.picorg\thumbs\`:

```
<id>-small.webp
<id>-medium.webp
```

Using the image id (rather than a hash of the source path) keeps the cache invariant across file renames: as long as PicOrg still knows about the image, the thumbnail is reusable.

Generation

```
pub fn ensure_thumbnails(cache_dir: &Path, src: &Path, image_id: i64) ->
Result<()> {
    let src_mtime = fs::metadata(src)?.modified()?;
    for size in &[ThumbSize::Small, ThumbSize::Medium] {
        let out = thumb_path(cache_dir, image_id, *size);
        if is_fresh(&out, src_mtime) { continue; }
        let img = image::open(src)?;
        let target = pick_target_size(&img, *size);
        let thumb = resize_rgba(&img.to_rgba8(), target);
        save_webp(&thumb, &out)?;
    }
    Ok(())
}
```

Details:

- `image::open` handles JPEG, PNG, GIF, TIFF, WebP, HEIC (with the `image` crate's HEIC feature enabled). For unsupported RAW formats, `open` returns an error and the thumbnail is skipped (grid renders a placeholder).
- `resize_rgba` uses `fast_image_resize::Resizer` with the `Lanczos3` filter. On x86_64 this dispatches to SSE4/AVX2 SIMD automatically.
- `save_webp` writes to `<out>.tmp` then renames — atomic, so a crash mid-encode never leaves a corrupt WebP.

Freshness

A thumbnail is considered fresh iff its file mtime is ≥ the source file's mtime. Editing metadata does not change the source mtime enough to invalidate a thumbnail (unless PicOrg's embed-XMP write touched the JPEG — in which case the thumbnail regenerates on next scan, which is correct because the JPEG has actually changed).

Deletion

`delete_thumbnails(cache_dir, image_id)` is called from `delete_images`. It removes every size:

```
for size in [Small, Medium, Large] {
    let _ = fs::remove_file(thumb_path(cache_dir, image_id, size));
}
```

Errors are ignored — a missing thumb is the desired state.

Cache sizing

The `Small` WebP averages 4–8 KB per photo; `Medium` averages 16–32 KB. Total cache size on a 50 000-photo library is roughly 1–2 GB.

There's no automatic eviction in v1. Users who want to trim the cache can delete `%APPDATA%\com.picorg.picorg\thumbs\` — PicOrg regenerates on demand.

Async model

`ensure_thumbnails` is CPU-bound (decode + resize + encode). It's called from Tauri commands via `tauri::async_runtime::spawn_blocking` so it doesn't stall a Tokio worker. The scanner enqueues thumb tasks on the rayon pool to overlap I/O with compute.

Metadata read path

Entry point

```
pub fn read_all(path: &Path) -> PicOrgResult<ImageMeta>;
```

Called from:

- The scanner (initial scan and rescans), to populate the DB.
- `get_image` (from a Tauri command), when a FS-refresh check detects that the source file (or any legacy sidecar) has changed since the DB last saw it.

`ImageMeta` is the union of everything we can pull from disk:

```
pub struct ImageMeta {
    pub title:        Option<String>,
    pub comment:      Option<String>,   // dc:description
    pub rating:       Option<i64>,      // 0..=5
    pub tags:         Vec<String>,      // deduped, in insertion order
    pub taken_at:     Option<i64>,      // ms since Unix epoch
    pub camera_make:  Option<String>,
    pub camera_model: Option<String>,
    pub width:        Option<u32>,
    pub height:       Option<u32>,
}
```

Stages

1. EXIF

```
let exif = exif::Reader::new()
    .continue_on_error(true)
    .read_from_container(&mut BufReader::new(File::open(path)?))?;
```

Fields extracted:

EXIF tag	ImageMeta field
`DateTimeOriginal`	`taken_at`
`Make`	`camera_make`
`Model`	`camera_model`
`PixelXDimension` / EXIF or IFD dimensions	`width`
`PixelYDimension`	`height`

`taken_at` parsing handles both `YYYY:MM:DD HH:MM:SS` and ISO-8601 strings, and preserves the local time zone if `OffsetTimeOriginal` is present.

If the file has no EXIF (PNG without an `eXIf` chunk, for example), we fall back to `image::image_dimensions(path)?` for the pixel size and leave the timing fields NULL.

2. Embedded XMP

```
let embedded = xmp::extract_embedded_xmp(path).ok().flatten();
```

Two format-specific walkers exist:

- **JPEG (`.jpg .jpeg .jpe .jfif .jif`)** — walks APP segments looking for `http://ns.adobe.com/xap/1.0/\0` (see Metadata write path for the exact layout). Read cap of 2 MiB — enough to catch a big Explorer-written XMP with a thumbnail while still being cheap.
- **PNG (`.png`)** — walks chunks looking for an `iTXt` chunk whose keyword is `XML:com.adobe.xmp`.

Formats without an embedded-XMP walker (WebP, TIFF, HEIC, RAW, GIF, BMP) return `None`; extending them is on the roadmap.

3. Legacy sidecar XMP

```
let sidecar = read_sidecar(&sidecar_path_for(path)).ok();
```

Sidecar path is `<image basename>.xmp`. Sidecars are a **legacy compatibility** path: PicOrg no longer produces them, but older PicOrg installs and other tools (Lightroom, digiKam) did, and

users would lose data if we ignored them. If a sidecar file exists it's parsed with the same XMP parser used for embedded.

4. Merge

`apply_user_meta` is called first with the embedded XMP (if any), then with the sidecar XMP (if any). The later call overwrites any field the earlier call set. Result: **sidecar wins over embedded**.

Rationale: the sidecar (if it still exists) is by definition older than PicOrg's current write path, but on first scan after upgrade we must respect edits already stored there. The precedence rule becomes moot after the first save into the source file, because `write_metadata_to_source` deletes the sidecar as part of its cleanup step.

XMP parser

`parse_user_metadata(bytes) -> UserMetadata` is a hand-written streaming parser built on `quick_xml`. Uses a simple state machine:

```
Idle → SubjectBag → SubjectItem → SubjectBag → SubjectItem → …
     ↘ MsKeywordBag → MsKeywordItem → …
     ↘ Title → (li text captured) → Idle
     ↘ Description → (li text captured) → Idle
```

- Recognises both element form (`<xmp:Rating>4</xmp:Rating>`) and attribute form (`xmp:Rating="4"` on `<rdf:Description>`).
- Recognises both `dc:subject` (standard) and `MicrosoftPhoto:LastKeywordXMP` (Windows Explorer). Unions the two sets, deduping case-insensitively while preserving the first observed casing.
- Handles both `<rdf:Alt>` and `<rdf:Seq>` inside `<dc:title>` and `<dc:description>` ; the `x-default` language item wins if present, otherwise the first item.

Everything is `<xmlns>` and `<case>` insensitive because tools in the wild are notoriously inconsistent (I've seen `<X:XMPMETA>` , `<x:xmpmeta>` , and mixed-case in the same file). See the `read_sidecar_case_variants` test.

FS-refresh check

Inside `get_image`, before returning, we call:

```
fn refresh_needed_from_fs(image_path: &Path, cached: &ImageDetails) -> bool {
    let last_read = cached.meta_read_at.unwrap_or(0);
    let src_mtime = mtime_ms(image_path);
    let sidecar_mtime = mtime_ms(&sidecar_path_for(image_path));
    src_mtime > last_read || sidecar_mtime > last_read
}
```

If it returns `true`, we call `read_all`, `resync_user_meta_from_fs` (which updates the DB row), and `set_meta_read_at_now` — then serve the fresh state.

We still watch the legacy sidecar's mtime here because a user might have run Lightroom against the same folder between two PicOrg scans, updating a `.xmp` without touching the source. Once PicOrg saves, the sidecar is cleaned up and the source-file mtime becomes the sole trigger.

This is how an external tag edit (Windows Explorer) becomes visible in PicOrg on next click, without needing a full library rescan.

Metadata write path

Entry point

```
pub fn write_metadata_to_source(
    image_path: &Path,
    patch_title:       Option<Option<String>>,
    patch_description: Option<Option<String>>,
    patch_rating:      Option<Option<i64>>,
    patch_subjects:    Option<Vec<String>>,
) -> PicOrgResult<()>;
```

Called from `apply_patch_and_persist` in `commands/images.rs`, which is itself called by both `update_image_metadata` and `batch_update_metadata`.

Policy. All user metadata is embedded directly inside the source image file. PicOrg never creates `.xmp` sidecar files. For formats we can't embed into today, `write_metadata_to_source` returns `PicOrgError::MetadataWrite` so the UI can surface a clear "unsupported format" toast to the user.

Supported formats

Extension	Writer
`.jpg .jpeg .jpe .jfif .jif`	JPEG APP1 XMP segment
`.png`	PNG `iTXt` chunk (keyword `XML:com.adobe.xmp`)
everything else	rejected with `MetadataWrite` error

The `format_supports_embedded_xmp(ext)` helper (in `core/metadata/xmp.rs`) is the single source of truth. The write function short-circuits on unsupported extensions *before* touching the disk, so a save on a `.cr2` never rewrites any bytes.

Algorithm

1. **Format gate.** If `format_supports_embedded_xmp(ext)` is false, return `Err(MetadataWrite("'ext' files can't store PicOrg metadata inside the file. Supported formats: JPEG, PNG."))`.
2. **Read existing.** `meta_read::read_all(image_path)` returns the current on-disk state (or `Err` — we treat that as an empty baseline).
3. **Apply patch.** For each of the four fields, if the patch has `Some(_)`, overwrite; otherwise keep the current value.
4. **Build packet.** `build_xmp_packet(&UserMetadata)` produces a full XMP packet as a `String`.
5. **Embed in source.** `embed_xmp_in_source(image_path, xmp_bytes)` dispatches to `embed_xmp_in_jpeg` or `embed_xmp_in_png`. Both are atomic (temp file next to the original, `sync_all`, rename).
6. **Delete legacy sidecar** (best effort). If a `<image>.xmp` from a previous PicOrg version or from Lightroom exists, remove it. Failure here is logged but does not fail the save — the source file already has the metadata.
7. **Log outcome.** `tracing::info!(?image_path, "embedded XMP into source file")`.

Any failure in steps 1, 2, 5 returns `Err(_)` to the caller. The DB update happens *before* `write_metadata_to_source` runs, so on file-write failure the DB is left with the user's edit (they don't lose their typing) but the UI's `onError` toast fires so they know the change didn't reach disk.

JPEG segment writer

```
fn embed_xmp_in_jpeg(path: &Path, xmp_bytes: &[u8]) -> PicOrgResult<()>;
```

Algorithm:

1. Read the whole file (JPEGs are small enough — a 20 MB photo is negligible).
2. Validate `[0xFF, 0xD8]` (SOI).
3. Build a new APP1 segment:
   - `0xFF 0xE1` marker.
   - `u16` big-endian length (2 + marker + xmp).
   - `http://ns.adobe.com/xap/1.0/\0` header.
   - The XMP packet bytes verbatim.
4. Walk the file scanning marker segments:
   - Standalone markers (SOI, EOI, RSTn) are copied.

    - Length-prefixed segments (APP0, DQT, DHT, SOF, …) are copied verbatim **except** APP1 segments whose payload starts with the standard XMP marker or the ExtendedXMP marker — those are dropped.
    - On SOS (`0xFFDA`), copy the rest of the file (compressed image data + EOI) without further parsing.
5. Emit `[SOI][new XMP APP1][…other segments…][SOS + data]`.
6. Write atomically to `<original>.picorg-tmp` and rename over the source.

Adobe's XMP specification recommends the standard XMP APP1 be the first APP1 after SOI, which is exactly where we put it.

ExtendedXMP

If the XMP packet is larger than 65533 bytes it can't fit in a single APP1 segment. Adobe's ExtendedXMP mechanism splits it across an "extended" segment with its own marker (`http://ns.adobe.com/xmp/extension/\0`). PicOrg's writer errors out in this case:

```
if payload_len > JPEG_MAX_SEGMENT_PAYLOAD {
    return Err(PicOrgError::MetadataWrite("XMP too large".into()));
}
```

In practice our packets are always under 4 KB. If we ever hit the limit (e.g. a photo with 500 tags), the reader side already supports ExtendedXMP; writing support would be a small extension.

PNG chunk writer

```
fn embed_xmp_in_png(path: &Path, xmp_bytes: &[u8]) -> PicOrgResult<()>;
```

PNG stores XMP in an `iTXt` (International Text) chunk with a well-known keyword. The exact layout:

```
chunk length  : u32 big-endian    ← length of data (excluding CRC)
chunk type    : "iTXt"            ← 4 bytes
data:
  keyword     : "XML:com.adobe.xmp" \0
  comp flag   : 0x00                ← uncompressed
  comp method: 0x00
  lang tag    : \0                    ← empty
  translated  : \0                    ← empty
  text        : <XMP packet bytes verbatim>
CRC           : u32 big-endian     ← CRC-32 of (type + data)
```

Algorithm:

1. Read the whole file.
2. Validate the 8-byte PNG signature (`89 50 4E 47 0D 0A 1A 0A`).
3. Walk chunks, copying each verbatim to the output **except**:
   - Any existing `iTXt` whose keyword equals `XML:com.adobe.xmp` is dropped.
   - Immediately after emitting `IHDR` (which the PNG spec requires to come first), emit our new XMP `iTXt` chunk. This places the metadata as early as possible in the file so viewers see it before decoding pixels.
4. Compute CRC-32 (polynomial `0xEDB88320` , reflected IEEE 802.3 — same as `crc32fast` , `flate2` , etc.) over `type + data` and append it to the new chunk.
5. Write the assembled bytes to `<original>.picorg-tmp` , then atomically rename over the source.

The CRC table is built once via `std::sync::OnceLock` and reused.

If the file has no IHDR chunk at all, we treat it as corrupt and return `Err(MetadataWrite("PNG has no IHDR chunk"))` rather than producing a file that other readers might reject.

Atomic file replace

Both writers funnel through `atomic_write_bytes` :

```
fn atomic_write_bytes(path: &Path, bytes: &[u8]) -> PicOrgResult<()> {
    let tmp = path.with_file_name(
        format!("{}.picorg-tmp", path.file_name().unwrap().to_string_lossy())
    );
    { let mut f = File::create(&tmp)?; f.write_all(bytes)?; f.sync_all().ok(); }
    fs::rename(&tmp, path).or_else(|e| { let _ = fs::remove_file(&tmp); Err(e)
})?;
    Ok(())
}
```

Windows semantics: `fs::rename` uses `MoveFileExW` internally with `MOVEFILE_REPLACE_EXISTING`, so the atomic swap works even when the target exists. On crash mid-write, the original file is still intact; at worst a `.picorg-tmp` file is left behind and is overwritten on the next successful write.

XMP packet builder

`build_xmp_packet(&UserMetadata) -> String` writes:

```
<?xpacket begin="" id="W5M0MpCehiHzreSzNTczkc9d"?>
<x:xmpmeta xmlns:x="adobe:ns:meta/" x:xmptk="PicOrg">
  <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-syntax-ns#">
    <rdf:Description rdf:about=""
        xmlns:xmp="http://ns.adobe.com/xap/1.0/"
        xmlns:dc="http://purl.org/dc/elements/1.1/"
        xmp:Rating="4"
        xmp:MetadataDate="2026-08-17T22:38:11+03:00">

      <dc:title>
        <rdf:Alt>
          <rdf:li xml:lang="x-default">Sunset over Vík</rdf:li>
        </rdf:Alt>
      </dc:title>

      <dc:description>
        <rdf:Alt>
          <rdf:li xml:lang="x-default">Long exposure, ND1000.</rdf:li>
        </rdf:Alt>
      </dc:description>

      <dc:subject>
        <rdf:Bag>
          <rdf:li>Iceland</rdf:li>
          <rdf:li>sunset</rdf:li>
        </rdf:Bag>
      </dc:subject>

    </rdf:Description>
  </rdf:RDF>
</x:xmpmeta>
<?xpacket end="w"?>
```

Empty fields are omitted (e.g. no title → no `<dc:title>` element). `xmp:MetadataDate` is set to the current time in RFC 3339.

Every text value is XML-escaped by `xml_escape(s)` before insertion so ampersands, angle brackets, and quotes never break the packet.

Legacy sidecar cleanup

After a successful embed, if `sidecar_path_for(image_path)` exists, `write_metadata_to_source` removes it:

```
let sidecar = sidecar_path_for(image_path);
if sidecar.exists() {
    if let Err(e) = std::fs::remove_file(&sidecar) {
        tracing::warn!(?sidecar, error = %e, "could not remove legacy sidecar");
    }
}
```

The deletion is best-effort: if removal fails (e.g. the sidecar is open in another editor), the save still returns `Ok(())` because the source file now holds the authoritative data — the stale sidecar will simply be overridden by the fresh embedded packet on the next read (or reaped on the next save).

Failure modes and recovery

Failure	Effect
Unsupported extension (RAW, HEIC, TIFF, ...)	Format gate returns `Err(_)` before any I/O. Nothing on disk.
DB transaction fails	Nothing is written to disk; caller receives `Err(_)`.
Source file read fails	Return `Err(_)`. DB update from earlier is kept (retryable).
Source file rewrite fails (tmp / rename)	Temp file removed if possible; original intact; caller sees `Err`.
Embed succeeds; sidecar delete fails	Save reports `Ok(())`; warning is logged; stale sidecar will be
	overridden by fresh embedded packet on next read.
Crash mid-write (any step)	Original file intact (atomic rename); temp file leaks and is
	eventually cleaned up on next successful write of same file.

Test coverage

Integration tests in `src-tauri/tests/metadata_fs.rs`:

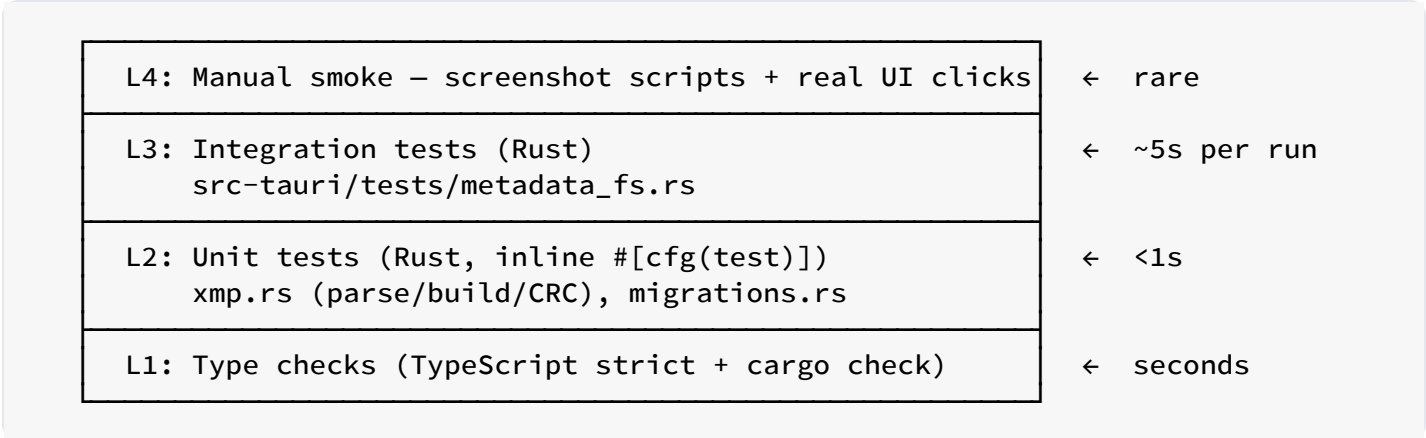
- `read_sidecar_end_to_end` — legacy sidecar read: title, description, rating, tags all recovered.
- `read_sidecar_case_variants` — namespace/case robustness.
- `fts_delete_after_tag_update_works` — regression for the FTS5 bug.
- `batch_tag_add_persists_for_every_image` — batch semantics.
- `embed_xmp_roundtrip_jpeg` — build a minimal JPEG, embed XMP, extract it back, verify content and JPEG validity.
- `embed_xmp_roundtrip_png` — same round-trip for PNG, plus verifies exactly one `iTXt` XMP chunk after a second write (no stacking).
- `write_never_creates_sidecar_for_jpeg` — writing to a JPEG embeds the packet and creates **zero** `.xmp` files.
- `write_removes_legacy_sidecar_after_embed` — an existing legacy sidecar is deleted after the source-file save.
- `write_errors_on_unsupported_format` — `.cr2` (and similar) get a clear error, no sidecar fallback.

Unit tests in `core::metadata::xmp::tests`:

- `roundtrip_basic`, `windows_explorer_tags`, `microsoft_only_keywords`.

Testing strategy

Layers



L1 — type checks

- `cargo check --workspace` for the Rust side (no unused warnings allowed in CI).
- `tsc -b --pretty false` for the frontend; strict mode enabled in `tsconfig.json` (`noUncheckedIndexedAccess`, `noImplicitAny`, `strictNullChecks`).

Both are cheap and run before every commit.

L2 — Rust unit tests

Colocated with modules in `#[cfg(test)] mod tests { … }`. Examples:

- `core/metadata/xmp.rs` — parser tests for Windows Explorer tags, Microsoft-only keyword lists, case variants, attribute-form values. Also covers the JPEG APP1 walker and (in integration tests) the PNG iTXt CRC.
- `db/queries.rs` — smoke tests for FTS row rebuild.

Run with:

```
cd src-tauri
cargo test --lib
```

L3 — Rust integration tests

`src-tauri/tests/metadata_fs.rs` exercises the full pipeline on temporary directories:

Test	Verifies
`read_sidecar_end_to_end`	Legacy sidecar read: title, description, rating, tags.
`read_sidecar_case_variants`	XMP parser is namespace/case insensitive.
`fts_delete_after_tag_update_works`	The FTS5 `contentless_delete` regression is fixed.
`batch_tag_add_persists_for_every_image`	Bulk `tags_add` / `tags_remove` semantics.
`embed_xmp_roundtrip_jpeg`	JPEG embed writes, re-reads correctly, replaces on second write.
`embed_xmp_roundtrip_png`	PNG iTXt embed writes, re-reads correctly, no chunk stacking on rewrite.
`write_never_creates_sidecar_for_jpeg`	Saving on a JPEG creates zero `.xmp` files.
`write_removes_legacy_sidecar_after_embed`	A pre-existing `.xmp` is removed on successful embed.
`write_errors_on_unsupported_format`	Saving on RAW returns `Err`, no sidecar fallback.

Run:

```
cd src-tauri
cargo test --test metadata_fs
```

L4 — screenshot smoke tests

Under `scripts/`:

- `screenshot-app.ps1` — launches PicOrg, finds its window with `EnumWindows`, screenshots via `PrintWindow` (`PW_RENDERFULLCONTENT`) so the shot works even if another window is in front.
- `screenshot-scrolled.ps1` — clicks an image, scrolls the details panel, screenshots.
- `test-multiselect-tag.ps1` — end-to-end skeleton for Ctrl+click → type tag → click Apply. UI automation is inherently flaky on Windows; use these mostly to verify a build launches and paints, not for asserting behaviour.

These are not part of automated CI. They're one-off tools when investigating a UI-only bug.

Frontend tests

Currently minimal:

- Type safety is the main safety net.
- Component tests would use Vitest + React Testing Library; not set up in v1.

Because the UI is a thin translation over Rust commands, the interesting behaviour is on the Rust side and covered by L2/L3.

Diagnostic tools (not tests, but adjacent)

- `src-tauri/examples/dump_meta.rs` — prints filesystem state, embedded XMP, any legacy sidecar contents, parsed metadata, and DB state for one photo or the first N rows in the DB. Great for reproducing bugs.
- `src-tauri/examples/dump_tag_usage.rs` — given a tag name, list every photo carrying it plus what the on-disk embedded XMP shows. Used in the "did the batch save actually work?" investigation.

Run:

```
cd src-tauri
cargo run --example dump_meta -- "C:\Photos\IMG_1234.jpg"
```

CI (recommended)

Not shipped in the repo yet, but the natural setup is:

- Windows runner (matrix: `windows-2022`).
- Steps: `cargo fmt --check`, `cargo clippy --all-targets`, `cargo test --workspace`, `npm ci`, `tsc -b`, `npm run tauri build -- --no-bundle`.
- Artifacts: `picorg.exe` for smoke inspection.

Build and release

Prerequisites (Windows)

```
# Rust toolchain (user scope)
winget install --id Rustlang.Rustup --accept-source-agreements

# MSVC C++ Build Tools (machine scope; needs UAC)
winget install --id Microsoft.VisualStudio.2022.BuildTools `
  --override "--wait --passive --add Microsoft.VisualStudio.Workload.VCTools --
includeRecommended"

# Node LTS
winget install --id OpenJS.NodeJS.LTS
```

Verify:

```
rustc --version   # ≥ 1.77.2
cargo --version
node --version    # ≥ 18
npm --version
```

Clone and build

```
git clone <repo> picorg
cd picorg
npm install
```

Development

```
npm run tauri dev
```

This starts:

- **Vite dev server** on `localhost:1420` with HMR.
- **Cargo watch** compiling the Rust core.
- **Tauri window** loading the dev server URL.

Rust code changes trigger a full app rebuild (10–30 s); frontend changes hot-reload in <1 s.

Release build

```
npm run tauri build -- --no-bundle
```

- `--no-bundle` skips the MSI/NSIS installer, producing just `src-tauri/target/release/picorg.exe`. Faster and useful for local testing.
- Remove `--no-bundle` (or pass `--bundles msi` / `--bundles nsis`) to produce an installer under `src-tauri/target/release/bundle/`.

Timings on a modern laptop:

- Cold release build: ~5 min.
- Incremental release build after a small Rust change: ~90 s.
- Incremental after frontend change only: ~30 s.

Tauri configuration

Key fields in `src-tauri/tauri.conf.json`:

Field	Notes
`productName`	`PicOrg`
`identifier`	`com.picorg.picorg` — used for `%APPDATA%` folder name.
`security.csp`	Strict; blocks inline scripts and remote origins.
`app.withGlobalTauri`	`false` — commands accessed via `@tauri-apps/api` only.
`bundle.icon`	Multi-size Windows ICO.
`bundle.windows.wix`	(Present in future for signed MSI.)

Capabilities (`src-tauri/capabilities/default.json`) declare only what's needed: dialog and opener plugins, asset access under the user's library folders. Network is *not* granted.

Release profile

`src-tauri/Cargo.toml`:

```
[profile.release]
codegen-units = 16
lto           = false
opt-level     = 3
panic         = "abort"
strip         = true
incremental   = false
```

Rationale:

- `codegen-units = 16` + `lto = false` — trades a bit of runtime perf for much faster incremental builds. In practice, the hot paths are limited by `image` decode and disk I/O, not by cross-crate inlining.
- `panic = "abort"` — smaller binary, no unwinding tables. All panics are treated as bugs; the app crashes and restarts.
- `strip = true` — removes debug symbols to keep the binary small (~13 MB stripped vs. 45 MB with symbols).

Version bump

1. `Cargo.toml`: `version = "0.1.1"` (both files if we ever add a workspace).
2. `package.json`: same.
3. `tauri.conf.json`: `version = "0.1.1"`.
4. Commit as `chore: v0.1.1`.

Releasing

The v1 workflow is manual:

1. `npm run tauri build` (with default bundle) on a clean tree.
2. Test the resulting MSI on a clean VM.
3. Draft a GitHub release with the MSI attached.
4. Attach the two PDFs from `docs/` as documentation assets.
5. Tag the commit `v0.1.0` and publish.

A GitHub Actions workflow is not shipped in v1 but can plug into the recommended CI setup in Testing strategy.